

Command-Based Programming

Command-Based 编程

中英对照 · Java 文档 · 11 篇文章

Contents / 目录

- 01 What Is “Command-Based” Programming? / 什么是 “Command-Based” 编程?
- 02 Commands / Command
- 03 Command Compositions / Command 组合
- 04 Subsystems / Subsystem
- 05 Binding Commands to Triggers / 将 Command 绑定到 Trigger
- 06 Structuring a Command-Based Robot Project / Command-Based 机器人项目的结构
- 07 Organizing Command-Based Robot Projects / 组织 Command-Based 机器人项目
- 08 The Command Scheduler / Command 调度器
- 09 PID Control in Command-based / Command-Based 中的 PID 控制
- 10 Motion Profiling in Command-based / Command-Based 中的运动曲线规划
- 11 Combining Motion Profiling and PID in Command-Based / 在 Command-Based 中结合运动曲线规划与 PID

01 What Is “Command-Based” Programming? / 什么是 “Command-Based” 编程?

WPILib supports a robot programming methodology called “command-based” programming. In general, “command-based” can refer both the general programming paradigm, and to the set of WPILib library resources included to facilitate it.

WPILib 支持一种称为 “Command-Based” 编程的机器人编程方法。一般而言, “Command-Based” 既可以指这种编程范式, 也可以指 WPILib 中为支持该范式而提供的一组库资源。

“Command-based” programming is one possible [design pattern](#) for robot software. It is not the only way to write a robot program, but it is a very effective one. Command-based robot code tends to be clean, extensible, and (with some tricks) easy to reuse from year to year.

“Command-Based” 编程是机器人软件可采用的一种设计模式。它并非编写机器人程序的唯一方式, 但非常有效。Command-Based 机器人代码通常清晰、易于扩展, 而且 (借助一些技巧) 易于逐年复用。

The command-based paradigm is also an example of [declarative programming](#). The command-based library allow users to define desired robot behaviors while minimizing the amount of iteration-by-iteration robot logic that they must write. For example, in the command-based program, a user can specify that “the robot should perform an action when a condition is true” (note the use of a [lambda](#)):

Command-Based 范式也是声明式编程的一个例子。Command-Based 库让用户定义期望的机器人行为, 同时尽量减少必须自行编写的逐周期机器人逻辑。例如, 在 Command-Based 程序中, 用户可以指定 “当某个条件为真时, 机器人应执行一个动作” (注意这里使用了 lambda) :

```
new Trigger(condition::get).onTrue(Commands.runOnce(() -> piston.set(DoubleSolenoid.Value.kForward)));
```

In contrast, without using command-based, the user would need to check the button state every iteration, and perform the appropriate action based on the state of the button.

相比之下, 如果不使用 Command-Based, 用户就需要在每个周期检查按钮状态, 并根据该状态执行相应动作。

```
if(condition.get()) {  
    if(!pressed) {  
        piston.set(DoubleSolenoid.Value.kForward);  
        pressed = true;  
    }  
} else {  
    pressed = false;  
}
```

Subsystems and Commands / subsystem 与 Command

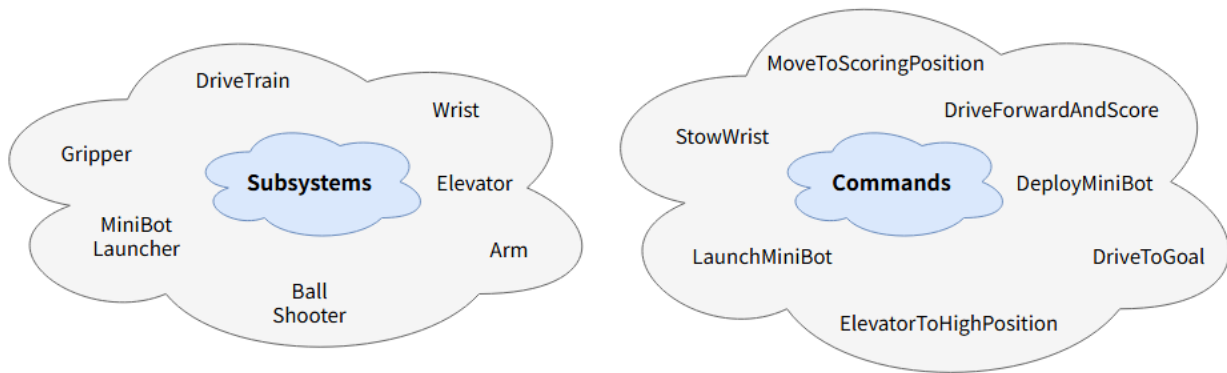


image of subsystems and commands / subsystem 与 Command 的示意图

The command-based pattern is based around two core abstractions: commands, and subsystems.

Command-Based 模式围绕两个核心抽象构建：Command 和 subsystem。

Commands represent actions the robot can take. Commands run when scheduled, until they are interrupted or their end condition is met. Commands are very recursively composable: commands can be composed to accomplish more-complicated tasks. See [Commands](#) for more info.

Command 表示机器人可以执行的动作。Command 被调度后开始运行，直到被中断或满足结束条件。

Command 非常适合递归组合：多个 Command 可以组合起来完成更复杂的任务。更多信息请参阅 [Commands](#)。

Subsystems represent independently-controlled collections of robot hardware (such as motor controllers, sensors, pneumatic actuators, etc.) that operate together. Subsystems back the resource-management system of command-based: only one command can use a given subsystem at the same time. Subsystems allow users to “hide” the internal complexity of their actual hardware from the rest of their code - this both simplifies the rest of the robot code, and allows changes to the internal details of a subsystem’s hardware without also changing the rest of the robot code.

Subsystem 表示一组相互配合、可独立控制的机器人硬件（例如电机控制器、传感器和气动执行器）。Subsystem 是 Command-Based 资源管理机制的基础：同一时间只能有一个 Command 使用指定的 subsystem。Subsystem 使用户可以向其余代码“隐藏”实际硬件的内部复杂性；这既简化了机器人的其余代码，也允许在不修改其余代码的情况下调整 subsystem 硬件的内部细节。

How Commands Are Run / Command 如何运行

NOTE: For a more detailed explanation, see [The Command Scheduler](#).

说明：更详细的说明请参阅 [The Command Scheduler](#)。

Commands are run by the [CommandScheduler](#) ([Java](#)) singleton, which polls triggers (such as buttons) for commands to schedule, preventing resource conflicts, and executing scheduled commands. The scheduler’s `run()` method must be called; it is generally recommended to call it from the `robotPeriodic()` method of the `Robot` class, which is run at a default frequency of 50Hz (once every 20ms).

Command 由 [CommandScheduler](#) 单例运行。该调度器轮询触发器（例如按钮）以调度 Command、防止资源冲突，并执行已调度的 Command。必须调用调度器的 `run()` 方法；通常建议在 `Robot` 类的 `robotPeriodic()`

) 方法中调用它。该方法默认以 50Hz 的频率运行（每 20ms 一次）。

Multiple commands can run concurrently, as long as they do not require the same resources on the robot. Resource management is handled on a per-subsystem basis: commands specify which subsystems they interact with, and the scheduler will ensure that no more more than one command requiring a given subsystem is scheduled at a time. This ensures that, for example, users will not end up with two different pieces of code attempting to set the same motor controller to different output values.

只要多个 Command 不需要机器人上的同一资源，它们就可以并发运行。资源管理以 subsystem 为单位：Command 指明自己与哪些 subsystem 交互，调度器确保同一时间至多有一个需要指定 subsystem 的 Command 被调度。例如，这可以避免两段不同代码同时试图为同一个电机控制器设置不同的输出值。

Command Compositions / Command 组合

It is often desirable to build complex commands from simple pieces. This is achievable by creating a [composition](#) of commands. The command-based library provides several types of [command compositions](#) for teams to use, and users may write their own. As command compositions are commands themselves, they may be used in a [recursive composition](#). That is to say - one can create a command compositions from multiple command compositions. This provides an extremely powerful way of building complex robot actions from simple components.

通常需要用简单的组成部分构建复杂 Command。可以通过创建 Command 的组合来实现这一点。Command-Based 库提供了几种供队伍使用的 Command 组合形式，用户也可以自行编写。由于 Command 组合本身也是 Command，因此也可以进行递归组合。也就是说，可以用多个 Command 组合再构成一个 Command 组合。这为从简单组件构建复杂机器人动作提供了非常有力的方式。

02 Commands / Command

Commands represent actions the robot can take. Commands run when scheduled, until they are interrupted or their end condition is met. Commands are represented in the command-based library by the `Command` class (Java).

Command 表示机器人可以执行的动作。Command 被调度后开始运行，直到被中断或满足结束条件。在 Command-Based 库中，Command 由 `Command` 类表示 (Java)。

The Structure of a Command / Command 的结构

Commands specify what the command will do in each of its possible states. This is done by overriding the `initialize()`, `execute()`, and `end()` methods. Additionally, a command must be able to tell the scheduler when (if ever) it has finished execution - this is done by overriding the `isFinished()` method. All of these methods are defaulted to reduce clutter in user code: `initialize()`, `execute()`, and `end()` are defaulted to simply do nothing, while `isFinished()` is defaulted to return false (resulting in a command that never finishes naturally, and will run until interrupted).

Command 指定它在各种可能状态下要执行的操作。这通过覆盖 `initialize()`、`execute()` 和 `end()` 方法实现。此外，Command 必须能够告知调度器它是否以及何时执行完毕；这通过覆盖 `isFinished()` 方法实现。所有这些方法都有默认实现，以减少用户代码中的冗余：`initialize()`、`execute()` 和 `end()` 默认什么也不做，而 `isFinished()` 默认返回 false（因此 Command 不会自然结束，会一直运行到被中断）。

Initialization / 初始化

The `initialize()` method (Java) marks the command start, and is called exactly once per time a command is scheduled. The `initialize()` method should be used to place the command in a known starting state for execution. Command objects may be reused and scheduled multiple times, so any state or resources needed for the command's functionality should be initialized or opened in `initialize` (which will be called at the start of each use) rather than the constructor (which is invoked only once on object allocation). It is also useful for performing tasks that only need to be performed once per time scheduled, such as setting motors to run at a constant speed or setting the state of a solenoid actuator.

`initialize()` 方法 (Java) 标志着 Command 的开始；每次 Command 被调度时恰好调用一次。应使用 `initialize()` 将 Command 设置为可执行的已知初始状态。Command 对象可以复用并被多次调度，因此其功能所需的状态或资源应在 `initialize` 中初始化或打开（每次使用开始时都会调用），而不是在构造函数中处理（分配对象时只调用一次）。它也适合执行每次调度只需进行一次的任务，例如让电机以恒定速度运行，或设置电磁阀执行器的状态。

Execution / 执行

The `execute()` method (Java) is called repeatedly while the command is scheduled; this is when the scheduler's `run()` method is called (this is generally done in the main robot periodic method, which runs every 20ms by default). The `execute` block should be used for any task that needs to be done continually while the command is scheduled, such as updating motor outputs to match joystick inputs, or using the output of a control loop.

Command 被调度期间，`execute()` 方法 (Java) 会在每次调用调度器 `run()` 方法时重复调用（通常是在机器人主周期方法中调用，默认每 20ms 一次）。`execute` 代码段应处理需要在 Command 被调度期间持续执行的任务，例如根据摇杆输入更新电机输出，或使用控制回路的输出。

Ending / 结束

The `end(boolean interrupted)` method (Java) is called once when the command ends, whether it finishes normally (i.e. `isFinished()` returned true) or it was interrupted (either by another command or by being explicitly canceled). The method argument specifies the manner in which the command ended; users can use this to differentiate the behavior of their command end accordingly. The end block should be used to “wrap up” command state in a neat way, such as setting motors back to zero or reverting a solenoid actuator to a “default” state. Any state or resources initialized in `initialize()` should be closed in `end()`.

当 Command 结束时，无论是正常完成（即 `isFinished()` 返回 true）还是被中断（由另一 Command 引起或被显式取消），`end(boolean interrupted)` 方法（Java）都会调用一次。方法参数指出 Command 的结束方式，用户可以据此区分结束时的行为。`end` 代码段应以整洁的方式收尾，例如将电机重新设为零输出，或使电磁阀执行器恢复到“默认”状态。任何在 `initialize()` 中初始化的状态或资源，都应在 `end()` 中关闭。

Specifying end conditions / 指定结束条件

The `isFinished()` method (Java) is called repeatedly while the command is scheduled, whenever the scheduler's `run()` method is called. As soon as it returns true, the command's `end()` method is called and it ends. The `isFinished()` method is called after the `execute()` method, so the command will execute once on the same iteration that it ends.

Command 被调度期间，每次调用调度器的 `run()` 方法时，都会重复调用 `isFinished()` 方法（Java）。它一旦返回 true，就会调用该 Command 的 `end()` 方法并结束 Command。`isFinished()` 在 `execute()` 之后调用，因此 Command 在结束的同一周期仍会执行一次。

Command Properties / Command 属性

In addition to the four lifecycle methods described above, each `Command` also has three properties, defined by getter methods that should always return the same value with no side affects.

除上述四个生命周期方法外，每个 `Command` 还有三个属性，由 getter 方法定义；这些方法应始终返回相同的值且没有副作用。

getRequirements / getRequirements

Each command should declare any subsystems it controls as requirements. This backs the scheduler's resource management mechanism, ensuring that no more than one command requires a given subsystem at the same time. This prevents situations such as two different pieces of code attempting to set the same motor controller to different output values.

每个 Command 都应将自己控制的 subsystem 声明为需求资源。这是调度器资源管理机制的基础，确保同一时间不会有多于一个 Command 需要同一 subsystem。这样可以避免两段不同代码试图给同一个电机控制器设置不同输出值等情况。

Declaring requirements is done by overriding the `getRequirements()` method in the relevant command class, by calling `addRequirements()`, or by using the `requirements` vararg (Java) parameter at the end of the parameter list of most command constructors and factories in the library:

可以在相关 Command 类中覆盖 `getRequirements()` 方法、调用 `addRequirements()`，或者使用库中大多数 Command 构造函数和工厂方法参数列表末尾的 `requirements` 可变参数（Java）来声明需求资源：

```
Commands.run(intake::activate, intake);
```

As a rule, command compositions require all subsystems their components require.

通常，Command 组合需要其所有组成 Command 所需的全部 subsystem。

runWhenDisabled / runWhenDisabled

The `runWhenDisabled()` method ([Java](#)) returns a `boolean` specifying whether the command may run when the robot is disabled. With the default of returning `false`, the command will be canceled when the robot is disabled and attempts to schedule it will do nothing. Returning `true` will allow the command to run and be scheduled when the robot is disabled.

`runWhenDisabled()` 方法 (Java) 返回一个 `boolean`，说明机器人禁用时是否允许该 Command 运行。默认返回 `false`；机器人禁用时，该 Command 会被取消，调度它的尝试也不会产生效果。返回 `true` 则允许该 Command 在机器人禁用时运行和被调度。

IMPORTANT: When the robot is disabled, [PWM](#) outputs are disabled and CAN motor controllers may not apply voltage, regardless of `runWhenDisabled`!

重要：机器人禁用时，[PWM](#) 输出会被禁用，CAN 电机控制器也可能不会施加电压，无论 `runWhenDisabled` 的值如何！

This property can be set either by overriding the `runWhenDisabled()` method in the relevant command class, or by using the `ignoringDisable` decorator ([Java](#)):

可以在相关 Command 类中覆盖 `runWhenDisabled()` 方法，或使用 `ignoringDisable` 装饰器 (Java) 设置这一属性：

```
Command mayRunDuringDisabled = Commands.run(() -> updateTelemetry()).ignoringDisable(true);
```

As a rule, command compositions may run when disabled if all their component commands set `runWhenDisabled` as `true`.

通常，仅当 Command 组合的所有组成 Command 都将 `runWhenDisabled` 设为 `true` 时，该组合才能在禁用状态下运行。

getInterruptBehavior / getInterruptBehavior

The `getInterruptBehavior()` method ([Java](#)) defines what happens if another command sharing a requirement is scheduled while this one is running. In the default behavior, `kCancelSelf`, the current command will be canceled and the incoming command will be scheduled successfully. If `kCancelIncoming` is returned, the incoming command's scheduling will be aborted and this command will continue running. Note that `getInterruptBehavior` only affects resolution of requirement conflicts: all commands can be canceled, regardless of `getInterruptBehavior`.

`getInterruptBehavior()` 方法 (Java) 定义了当前 Command 正在运行时，如果另一个具有相同需求资源的 Command 被调度，会发生什么。默认行为 `kCancelSelf` 会取消当前 Command，并成功调度新 Command。如果返回 `kCancelIncoming`，新 Command 的调度会中止，而当前 Command 继续运行。注意，`getInterruptBehavior` 仅影响需求资源冲突的解决方式：所有 Command 都可以被取消，无论其 `getInterruptBehavior` 是什么。

NOTE: This was previously controlled by the `interruptible` parameter passed when scheduling a command, and is now a property of the command object.

说明：过去，这由调度 Command 时传入的 `interruptible` 参数控制；现在它是 Command 对象的属性。

This property can be set either by overriding the `getInterruptBehavior` method in the relevant command class, or by using the `withInterruptBehavior()` decorator ([Java](#))

可以在相关 `Command` 类中覆盖 `getInterruptBehavior` 方法，或使用 `withInterruptBehavior()` 装饰器 (Java) 设置这一属性。

```
Command noninterruptible = Commands.run(intake::activate, intake).withInterruptBehavior(Command.InterruptBehavior.kCancelIncoming);
```

As a rule, command compositions are `kCancelIncoming` if all their components are `kCancelIncoming` as well.

通常，仅当 `Command` 组合的所有组成 `Command` 都是 `kCancelIncoming` 时，该组合才是 `kCancelIncoming`。

Included Command Types / 内置 Command 类型

The command-based library includes many pre-written command types. Through the use of [lambdas](#), these commands can cover almost all use cases and teams should rarely need to write custom command classes. Many of these commands are provided via static factory functions in the `Commands` utility class (Java). In Java, classes inheriting from `Subsystem` also have instance methods that implicitly require `this`.

Command-Based 库提供了许多预先编写好的 `Command` 类型。借助 `lambda`，这些 `Command` 几乎可以覆盖所有用例，队伍很少需要编写自定义 `Command` 类。许多 `Command` 通过 `Commands` 工具类中的静态工厂方法提供 (Java)。此外，继承 `Subsystem` 的类也有隐式地将 `this` 声明为需求资源的实例方法。

Running Actions / 运行动作

The most basic commands are actions the robot takes: setting voltage to a motor, changing a solenoid's direction, etc. For these commands, which typically consist of a method call or two, the command-based library offers several factories to be construct commands inline with one or more lambdas to be executed.

最基本的 `Command` 是机器人执行的动作，例如向电机设置电压、改变电磁阀方向等。对于这类通常只包含一两次方法调用的 `Command`，`Command-Based` 库提供了几个工厂方法，可使用一个或多个待执行的 `lambda` 直接构造 `Command`。

The `runOnce` factory, backed by the `InstantCommand` ([Java](#)) class, creates a command that calls a lambda once, and then finishes.

`runOnce` 工厂方法由 `InstantCommand` 类支持，创建一个调用 `lambda` 一次后便结束的 `Command`。

```
/** Grabs the hatch. */
public Command grabHatchCommand() {
    // implicitly require `this`
    return this.runOnce(() -> m_hatchSolenoid.set(kForward));
}
/** Releases the hatch. */
public Command releaseHatchCommand() {
    // implicitly require `this`
    return this.runOnce(() -> m_hatchSolenoid.set(kReverse));
}
```

The `run` factory, backed by the `RunCommand` ([Java](#)) class, creates a command that calls a lambda repeatedly, until interrupted.

`run` 工厂方法由 `RunCommand` 类支持，创建一个反复调用 `lambda`、直到被中断才结束的 `Command`。

```
// A split-stick arcade command, with forward/backward controlled by the left
// hand, and turning controlled by the right.
new RunCommand(() -> m_robotDrive.arcadeDrive(
    -driverController.getLeftY(),
    driverController.getRightX(),
    m_robotDrive)
```


The `startEnd` factory, backed by the `StartEndCommand` ([Java](#)) class, calls one lambda when scheduled, and then a second lambda when interrupted.

`startEnd` 工厂方法由 `StartEndCommand` 类支持，在被调度时调用第一个 lambda，在被中断时调用第二个 lambda。

```
Commands.startEnd(
    // Start a flywheel spinning at 50% power
    () -> m_shooter.shooterSpeed(0.5),
    // Stop the flywheel at the end of the command
    () -> m_shooter.shooterSpeed(0.0),
    // Requires the shooter subsystem
    m_shooter
)
```

`FunctionalCommand` ([Java](#)) accepts four lambdas that constitute the four command lifecycle methods: a `Runnable` for each of `initialize()` and `execute()`, a `BooleanConsumer` for `end()`, and a `BooleanSupplier` for `isFinished()`.

`FunctionalCommand` (Java) 接受四个 lambda，分别构成 Command 的四个生命周期方法：`initialize()` 和 `execute()` 各接收一个 `Runnable`，`end()` 接收一个 `BooleanConsumer`，`isFinished()` 接收一个 `BooleanSupplier`。

```
new FunctionalCommand(
    // Reset encoders on command start
    m_robotDrive::resetEncoders,
    // Start driving forward at the start of the command
    () -> m_robotDrive.arcadeDrive(kAutoDriveSpeed, 0),
    // Stop driving at the end of the command
    interrupted -> m_robotDrive.arcadeDrive(0, 0),
    // End the command when the robot's driven distance exceeds the desired value
    () -> m_robotDrive.getAverageEncoderDistance() >= kAutoDriveDistanceInches,
    // Require the drive subsystem
    m_robotDrive
)
```

To print a string and ending immediately, the library offers the `Commands.print(String)` factory, backed by the `PrintCommand` ([Java](#)) subclass of `InstantCommand`.

为了打印一个字符串并立即结束，库提供了 `Commands.print(String)` 工厂方法。它由 `PrintCommand` 类支持，而 `PrintCommand` 是 `InstantCommand` 的子类。

Waiting / 等待

Waiting for a certain condition to happen or adding a delay can be useful to synchronize between different commands in a command composition or between other robot actions.

等待某个条件出现或添加延时，可用于同步 Command 组合中的不同 Command，或同步其他机器人动作。

To wait and end after a specified period of time elapses, the library offers the `Commands.waitSeconds(double)` factory, backed by the `WaitCommand` ([Java](#)) class.

要等待指定时长后结束，库提供了由 `WaitCommand` 类支持的 `Commands.waitSeconds(double)` 工厂方法。

```
// Ends 5 seconds after being scheduled
new WaitCommand(5.0)
```

To wait until a certain condition becomes `true`, the library offers the `Commands.waitUntil(BooleanSupplier)` factory, backed by the `WaitUntilCommand` class ([Java](#)).

要等待某个条件变为 `true`，库提供了由 `WaitUntilCommand` 类支持的 `Commands.waitUntil(BooleanSupplier)` 工厂方法。

```
// Ends after m_limitSwitch.get() returns true
new WaitUntilCommand(m_limitSwitch::get)
```

Control Algorithm Commands / 控制算法 Command

There are commands for various control setups:

针对不同的控制场景，库提供了以下 Command：

- [TrapezoidProfile](#) tracks a trapezoid motion profile. For more info, see [Motion Profiling in Command-based](#).
[TrapezoidProfile](#) 跟踪梯形运动曲线。更多信息请参阅 [Motion Profiling in Command-Based](#).
- [MecanumControllerCommand](#) ([Java](#)) is useful for controlling mecanum drivetrains. See API docs and the [MecanumControllerCommand](#) ([Java](#)) example project for more info.
[MecanumControllerCommand](#) (Java) 适用于控制麦克纳姆底盘。更多信息请参阅 API 文档和 [MecanumControllerCommand](#) 示例项目 (Java)。
- [SwerveControllerCommand](#) ([Java](#)) is useful for controlling swerve drivetrains. See API docs and the [SwerveControllerCommand](#) ([Java](#)) example project for more info.
[SwerveControllerCommand](#) (Java) 适用于控制 swerve 底盘。更多信息请参阅 API 文档和 [SwerveControllerCommand](#) 示例项目 (Java)。
- [RamseteCommand](#) ([Java](#)) is useful for path following with differential drivetrains ("tank drive"). See API docs and the [Trajectory Tutorial](#) for more info.
[RamseteCommand](#) (Java) 适用于差速底盘 ("坦克底盘") 的路径跟踪。更多信息请参阅 API 文档和 [Trajectory Tutorial](#)。

Custom Command Classes / 自定义 Command 类

Users may also write custom command classes. As this is significantly more verbose, it' s recommended to use the more concise factories mentioned above.

用户也可以编写自定义 Command 类。由于这样做明显更冗长，推荐使用上面提到的简洁工厂方法。

To write a custom command class, subclass the abstract [Command](#) class ([Java](#)), as seen in the command-based template ([Java](#)):

要编写自定义 Command 类，可以继承抽象类 [Command](#) (Java)，如 [Command-Based 模板](#) (Java) 所示：

```
import edu.wpi.first.wpilibj.templates.commandbased.subsystems.ExampleSubsystem;
import edu.wpi.first.wpilibj2.command.Command;
/** An example command that uses an example subsystem. */
public class ExampleCommand extends Command {
    @SuppressWarnings("PMD.UnusedPrivateField")
    private final ExampleSubsystem m_subsystem;
    /**
     * Creates a new ExampleCommand.
     *
     * @param subsystem The subsystem used by this command.
     */
    public ExampleCommand(ExampleSubsystem subsystem) {
        m_subsystem = subsystem;
        // Use addRequirements() here to declare subsystem dependencies.
        addRequirements(subsystem);
    }
}
```

Simple Command Example / 简单 Command 示例

What might a functional command look like in practice? As before, below is a simple command from the HatchBot example project ([Java](#)) that uses the `HatchSubsystem`:

实际可用的 Command 会是什么样？与前面一样，下面是 HatchBot 示例项目（Java）中的一个简单 Command，它使用 `HatchSubsystem`：

```
package edu.wpi.first.wpilibj.examples.hatchbottraditional.commands;
import edu.wpi.first.wpilibj.examples.hatchbottraditional.subsystems.HatchSubsystem;
import edu.wpi.first.wpilibj2.command.Command;
/**
 * A simple command that grabs a hatch with the {@link HatchSubsystem}. Written explicitly for
 * pedagogical purposes. Actual code should inline a command this simple with {@link
 * edu.wpi.first.wpilibj2.command.InstantCommand}.
 */
public class GrabHatch extends Command {
    // The subsystem the command runs on
    private final HatchSubsystem m_hatchSubsystem;
    public GrabHatch(HatchSubsystem subsystem) {
        m_hatchSubsystem = subsystem;
        addRequirements(m_hatchSubsystem);
    }
    @Override
    public void initialize() {
        m_hatchSubsystem.grabHatch();
    }
    @Override
    public boolean isFinished() {
        return true;
    }
}
```

Notice that the hatch subsystem used by the command is passed into the command through the command's constructor. This is a pattern called [dependency injection](#), and allows users to avoid declaring their subsystems as global variables. This is widely accepted as a best-practice - the reasoning behind this is discussed in a [later section](#).

注意，Command 所使用的舱门 subsystem 通过该 Command 的构造函数传入。这一模式称为“依赖注入”，让用户无须将 subsystem 声明为全局变量。这被广泛视为良好实践；其原因将在后面的章节中讨论。

Notice also that the above command calls the subsystem method once from initialize, and then immediately ends (as `isFinished()` simply returns true). This is typical for commands that toggle the states of subsystems, and as such it would be more succinct to write this command using the factories described above.

还应注意，上述 Command 在 `initialize` 中调用 subsystem 方法一次，随后立即结束（因为 `isFinished()` 直接返回 true）。这对于切换 subsystem 状态的 Command 很常见，因此用上文介绍的工厂方法编写会更简洁。

What about a more complicated case? Below is a drive command, from the same example project:

更复杂的情况又如何？下面是同一个示例项目中的一个驱动 Command：

```
package edu.wpi.first.wpilibj.examples.hatchbottraditional.commands;
import edu.wpi.first.wpilibj.examples.hatchbottraditional.subsystems.DriveSubsystem;
import edu.wpi.first.wpilibj2.command.Command;
import java.util.function.DoubleSupplier;
/**
 * A command to drive the robot with joystick input (passed in as {@link DoubleSupplier}s). Written
 * explicitly for pedagogical purposes - actual code should inline a command this simple with {@link
 * edu.wpi.first.wpilibj2.command.RunCommand}.
 */
public class DefaultDrive extends Command {
    private final DriveSubsystem m_drive;
    private final DoubleSupplier m_forward;
    private final DoubleSupplier m_rotation;
    /**
     * Creates a new DefaultDrive.
     */
}
```

```

*
* @param subsystem The drive subsystem this command will run on.
* @param forward The control input for driving forwards/backwards
* @param rotation The control input for turning
*/
public DefaultDrive(DriveSubsystem subsystem, DoubleSupplier forward, DoubleSupplier rotation) {
    m_drive = subsystem;
    m_forward = forward;
    m_rotation = rotation;
    addRequirements(m_drive);
}
@Override
public void execute() {
    m_drive.arcadeDrive(m_forward.getAsDouble(), m_rotation.getAsDouble());
}
}

```

And then usage:

然后是其用法：

```

// Configure default commands
// Set the default drive command to split-stick arcade drive
m_robotDrive.setDefaultCommand(
    // A split-stick arcade command, with forward/backward controlled by the left
    // hand, and turning controlled by the right.
    new DefaultDrive(
        m_robotDrive,
        () -> -m_driverController.getLeftY(),
        () -> -m_driverController.getRightX()));

```

Notice that this command does not override `isFinished()`, and thus will never end; this is the norm for commands that are intended to be used as default commands. Once more, this command is rather simple and calls the subsystem method only from one place, and as such, could be more concisely written using factories:

注意，这个 Command 没有覆盖 `isFinished()`，因此永远不会自行结束；这正是用作默认 Command 时的常见做法。再者，这个 Command 相当简单，只在一个位置调用 `subsystem` 方法，因此也可以用工厂方法写得更简洁：

```

// Configure default commands
// Set the default drive command to split-stick arcade drive
m_robotDrive.setDefaultCommand(
    // A split-stick arcade command, with forward/backward controlled by the left
    // hand, and turning controlled by the right.
    Commands.run(
        () ->
            m_robotDrive.arcadeDrive(
                -m_driverController.getLeftY(), -m_driverController.getRightX()),
        m_robotDrive));

```

03 Command Compositions / Command 组合

Individual commands are capable of accomplishing a large variety of robot tasks, but the simple three-state format can quickly become cumbersome when more advanced functionality requiring extended sequences of robot tasks or coordination of multiple robot subsystems is required. In order to accomplish this, users are encouraged to use the powerful command composition functionality included in the command-based library.

单个 Command 可以完成多种机器人任务，但当需要较长的任务序列或协调多个机器人 subsystem 等高级功能时，简单的三状态格式很快就会变得繁琐。为此，建议用户使用 Command-Based 库提供的强大 Command 组合功能。

As the name suggests, a command composition is a [composition](#) of one or more commands. This allows code to be kept much cleaner and simpler, as the individual component commands may be written independently of the code that combines them, greatly reducing the amount of complexity at any given step of the process.

顾名思义，Command 组合由一个或多个 Command 组成。各个组成 Command 可以独立于组合它们的代码编写，因此代码能够保持更清晰、更简单，并显著降低过程各步骤中的复杂度。

Most importantly, however, command compositions are themselves commands - they extend the [Command](#) class. This allows command compositions to be further composed as a [recursive composition](#) - that is, a command composition may contain other command compositions as components. This allows very powerful and concise inline expressions:

更重要的是，Command 组合本身也是 Command——它们继承 [Command](#) 类。因此，Command 组合还可以进一步递归组合，也就是说，一个 Command 组合可以包含其他 Command 组合作为组成部分。这使得功能强大而简洁的内联表达式成为可能：

```
// Will run fooCommand, and then a race between barCommand and bazCommand
button.onTrue(fooCommand.andThen(barCommand.raceWith(bazCommand)));
```

As a rule, command compositions require all subsystems their components require, may run when disabled if all their component set `runsWhenDisabled` as `true`, and are `kCancelIncoming` if all their components are `kCancelIncoming` as well.

通常，Command 组合需要其组成 Command 所需的全部 subsystem；只有当所有组成 Command 的 `runsWhenDisabled` 都为 `true` 时，组合才可在禁用状态下运行；只有当所有组成 Command 都是 `kCancelIncoming` 时，组合才是 `kCancelIncoming`。

Command instances that have been passed to a command composition cannot be independently scheduled or passed to a second command composition. Attempting to do so will throw an exception and crash the user program. This is because composition members are run through their encapsulating command composition, and errors could occur if those same command instances were independently scheduled at the same time as the composition - the command would be being run from multiple places at once, and thus could end up with inconsistent internal state, causing unexpected and hard-to-diagnose behavior.

已传给某个 Command 组合的 Command 实例，不能再独立调度，也不能再传给第二个 Command 组合。尝试这样做会抛出异常并导致用户程序崩溃。这是因为组成 Command 会通过封装它们的组合运行；如果同一实例又被独立调度，就可能同时从多个位置运行，造成内部状态不一致，并导致意外且难以诊断的行为。

Composition Types / 组合类型

The command-based library includes various composition types. All of them can be constructed using factories that accept the member commands, and some can also be constructed using decorators: methods that can be called on a command object, which is transformed into a new object that is returned.

Command-Based 库提供了多种组合类型。所有组合都可通过接收组成 Command 的工厂方法构造，其中有些也能通过装饰器构造。装饰器是可以在 Command 对象上调用的方法，它会将该 Command 转换为一个新对象并返回。

IMPORTANT: After calling a decorator or being passed to a composition, the command object cannot be reused! Use only the command object returned from the decorator.

重要： 调用装饰器，或将 Command 对象传给组合之后，便不能复用原 Command 对象！只使用装饰器返回的 Command 对象。

Repeating / 重复

The `repeatedly()` decorator ([Java](#)), backed by the `RepeatCommand` class ([Java](#)) restarts the command each time it ends, so that it runs until interrupted.

`repeatedly()` 装饰器 (Java) 由 `RepeatCommand` 类支持；它每次在 Command 结束后重新启动该 Command，使其持续运行直到被中断。

```
// Will run forever unless externally interrupted, restarting every time command.isFinished() returns true
Command repeats = command.repeatedly();
```

Sequence / 顺序执行

The `Sequence` factory ([Java](#)), backed by the `SequentialCommandGroup` class ([Java](#)), runs a list of commands in sequence: the first command will be executed, then the second, then the third, and so on until the list finishes. The sequential group finishes after the last command in the sequence finishes. It is therefore usually important to ensure that each command in the sequence does actually finish (if a given command does not finish, the next command will never start!).

`Sequence` 工厂方法 (Java) 由 `SequentialCommandGroup` 类支持，依次运行 Command 列表：先执行第一个，再执行第二个、第三个，依此类推，直到列表结束。序列组在最后一个 Command 结束后才结束。因此，通常必须确保序列中的每个 Command 确实会结束（如果某个 Command 不结束，下一个 Command 就永远不会开始！）。

The `andThen()` ([Java](#)) and `beforeStarting()` ([Java](#)) decorators can be used to construct a sequence composition with infix syntax.

`andThen()` 和 `beforeStarting()` 装饰器 (Java) 可以用链式语法构造顺序组合。

```
fooCommand.andThen(barCommand)
```

Repeating Sequence / 重复序列

As it's a fairly common combination, the `RepeatingSequence` factory ([Java](#)) creates a Repeating Sequence that runs until interrupted, restarting from the first command each time the last command finishes.

由于这种组合相当常见，`RepeatingSequence` 工厂方法 (Java) 可以创建持续运行到被中断的重复序列；每当最后一个 Command 结束时，就从第一个 Command 重新开始。

Parallel / 并行执行

There are three types of parallel compositions, differing based on when the composition finishes:

并行组合有三种类型，区别在于组合何时结束：

- The `Parallel` factory ([Java](#)), backed by the `ParallelCommandGroup` class ([Java](#)), constructs a parallel composition that finishes when all members finish. The `alongWith` decorator ([Java](#)) does the same in infix notation.

`Parallel` 工厂方法 (Java) 由 `ParallelCommandGroup`

类支持，构造一个在所有成员都结束时才结束的并行组合。`alongWith`

装饰器 (Java) 可用链式语法实现同样的功能。

- The `Race` factory ([Java](#)), backed by the `ParallelRaceGroup` class ([Java](#)), constructs a parallel composition that finishes as soon as any member finishes; all other members are interrupted at that point. The `raceWith` decorator ([Java](#)) does the same in infix notation.

`Race` 工厂方法 (Java) 由 `ParallelRaceGroup`

类支持，构造一个在任意成员结束时立即结束的并行组合；此时会中断所有其他成员。`raceWith`

装饰器 (Java) 可用链式语法实现同样的功能。

- The `Deadline` factory ([Java](#)), `ParallelDeadlineGroup` ([Java](#)) finishes when a specific command (the “deadline”) ends; all other members still running at that point are interrupted. The `deadlineWith` decorator ([Java](#)) does the same in infix notation; the command the decorator was called on is the deadline.

`Deadline` 工厂方法 (Java) 由 `ParallelDeadlineGroup` 类支持，在指定 Command (即 “截止

Command”) 结束时结束；此时仍在运行的其他成员会被中断。`deadlineWith`

装饰器 (Java) 可用链式语法实现同样的功能；调用该装饰器的 Command 就是截止 Command。

```
// Will be a parallel command composition that ends after three seconds with all three commands running their full duration.
button.onTrue(Commands.parallel(twoSecCommand, oneSecCommand, threeSecCommand));
// Will be a parallel race composition that ends after one second with the two and three second commands getting interrupted.
button.onTrue(Commands.race(twoSecCommand, oneSecCommand, threeSecCommand));
// Will be a parallel deadline composition that ends after two seconds (the deadline) with the three second command getting interrupted (one second command already finished).
button.onTrue(Commands.deadline(twoSecCommand, oneSecCommand, threeSecCommand));
```

Adding Command End Conditions / 添加 Command 结束条件

The `until()` ([Java](#)) decorator composes the command with an additional end condition. Note that the command the decorator was called on will see this end condition as an interruption.

`until()` 装饰器 (Java) 为 Command 组合一个额外的结束条件。注意，被调用装饰器的原 Command 会将这个结束条件视为一次中断。

```
// Will be interrupted if m_limitSwitch.get() returns true
button.onTrue(command.until(m_limitSwitch::get));
```

The `withTimeout()` decorator ([Java](#)) is a specialization of `until` that uses a timeout as the additional end condition.

`withTimeout()` 装饰器 (Java) 是 `until` 的一种特例，用超时作为额外的结束条件。

```
// Will time out 5 seconds after being scheduled, and be interrupted
button.onTrue(command.withTimeout(5));
```


Adding End Behavior / 添加结束行为

The `finallyDo()` ([Java](#)) decorator composes the command with an a lambda that will be called after the command' s `end()` method, with the same boolean parameter indicating whether the command finished or was interrupted.

`finallyDo()` 装饰器 (Java) 为 Command 组合一个 lambda, 它会在该 Command 的 `end()` 方法之后调用, 并接收同一个布尔参数, 以说明 Command 是正常结束还是被中断。

The `handleInterrupt()` ([Java](#)) decorator composes the command with an a lambda that will be called only when the command is interrupted.

`handleInterrupt()` 装饰器 (Java) 为 Command 组合一个只在该 Command 被中断时调用的 lambda。

Selecting Compositions / 选择组合

Sometimes it' s desired to run a command out of a few options based on sensor feedback or other data known only at runtime. This can be useful for determining an auto routine, or running a different command based on whether a game piece is present or not, and so on.

有时, 希望根据传感器反馈或其他只有在运行时才知道的数据, 从几个选项中选择一个 Command 运行。这可以用于确定自动程序, 或者根据是否存在游戏物件运行不同的 Command 等情况。

The `Select` factory ([Java](#)), backed by the `SelectCommand` class ([Java](#)), executes one command from a map, based on a selector function called when scheduled.

`Select` 工厂方法 (Java) 由 `SelectCommand` 类支持; 它根据 Command 被调度时调用的选择函数, 从映射表中选择并执行一个 Command。

```
public class RobotContainer {
    // The enum used as keys for selecting the command to run.
    private enum CommandSelector {
        ONE,
        TWO,
        THREE
    }
    // An example selector method for the selectcommand. Returns the selector that will select
    // which command to run. Can base this choice on logical conditions evaluated at runtime.
    private CommandSelector select() {
        return CommandSelector.ONE;
    }
    // An example selectcommand. Will select from the three commands based on the value returned
    // by the selector method at runtime. Note that selectcommand works on Object(), so the
    // selector does not have to be an enum; it could be any desired type (string, integer,
    // boolean, double...)
    private final Command m_exampleSelectCommand =
        new SelectCommand<>() {
            // Maps selector values to commands
            Map.ofEntries(
                Map.entry(CommandSelector.ONE, new PrintCommand("Command one was selected!")),
                Map.entry(CommandSelector.TWO, new PrintCommand("Command two was selected!")),
                Map.entry(CommandSelector.THREE, new PrintCommand("Command three was selected!")),
            ),
            this::select;
        };
}
```

The `Either` factory ([Java](#)), backed by the `ConditionalCommand` class ([Java](#)), is a specialization accepting two commands and a boolean selector function.

`Either` 工厂方法 (Java) 由 `ConditionalCommand` 类支持, 是接收两个 Command 和一个布尔选择函数的特例。

```
// Runs either commandOnTrue or commandOnFalse depending on the value of m_limitSwitch.get()
new ConditionalCommand(commandOnTrue, commandOnFalse, m_limitSwitch::get)
```

The `unless()` decorator ([Java](#)) composes a command with a condition that will prevent it from running.

`unless()` 装饰器 (Java) 为 Command 组合一个可阻止其运行的条件。

```
// Command will only run if the intake is deployed. If the intake gets deployed while the command is running, the command will not
// stop running
button.onTrue(command.unless(() -> !intake.isDeployed()));
```

`ProxyCommand` described below also has a constructor overload ([Java](#)) that calls a command-returning lambda at schedule-time and runs the returned command by proxy.

下文介绍的 `ProxyCommand` 也有一个构造函数重载 (Java)，它会在调度时调用返回 Command 的 lambda，并通过代理运行返回的 Command。

Scheduling Other Commands / 调度其他 Command

By default, composition members are run through the command composition, and are never themselves seen by the scheduler. Accordingly, their requirements are added to the composition's requirements. While this is usually fine, sometimes it is undesirable for the entire command composition to gain the requirements of a single command. A good solution is to “fork off” from the command composition and schedule that command separately. However, this requires synchronization between the composition and the individually-scheduled command.

默认情况下，组成 Command 通过 Command 组合运行，调度器不会直接看到它们。因此，它们的需求资源会加入组合的需求资源。通常这没有问题，但有时不希望整个 Command 组合获得某一个 Command 的需求资源。一个可行办法是从组合中“分出”该 Command，将其单独调度。不过，这要求组合与单独调度的 Command 之间进行同步。

`ProxyCommand` ([Java](#)), also creatable using the `.asProxy()` decorator ([Java](#)), schedules a command “by proxy” : the command is scheduled when the proxy is scheduled, and the proxy finishes when the command finishes. In the case of “forking off” from a command composition, this allows the composition to track the command's progress without it being in the composition.

`ProxyCommand` (Java) 也可以通过 `.asProxy()` 装饰器 (Java) 创建。它会以“代理”方式调度一个 Command：代理被调度时，该 Command 也被调度；该 Command 结束时，代理也结束。对于从 Command 组合中“分出” Command 的情况，这使组合能跟踪 Command 的进度，而不必把该 Command 纳入组合。

Command compositions inherit the union of their components' requirements and requirements are immutable. Therefore, a `SequentialCommandGroup` ([Java](#)) that intakes a game piece, indexes it, aims a shooter, and shoots it would reserve all three subsystems (the intake, indexer, and shooter), precluding any of those subsystems from performing other operations in their “downtime”. If this is not desired, the subsystems that should only be reserved for the composition while they are actively being used by it should have their commands proxied.

Command 组合会继承其组成 Command 的需求资源并集，而且需求资源不可变。因此，一个负责吸入游戏物件、将其送入索引机构、瞄准发射器并发射的 `SequentialCommandGroup` (Java)，会占用全部三个 subsystem（吸取机构、索引机构和发射器），使它们在各自的“空闲阶段”也无法执行其他操作。如果不希望如此，应将那些只在被组合实际使用时才需要占用的 subsystem 所对应的 Command 设为代理。

WARNING: Do not use `ProxyCommand` unless you are sure of what you are doing and there is no other way to accomplish your need! Proxying is only intended for use as an escape hatch from command composition requirement unions.

警告：除非确定自己在做什么，而且没有其他方式满足需求，否则不要使用 `ProxyCommand`！代理仅用于作为摆脱 Command 组合需求资源并集限制的备用手段。

NOTE: Because proxied commands still require their subsystem, despite not leaking that requirement to the composition, all of the commands that require a given subsystem must be proxied if one of them is. Otherwise, when the proxied command is scheduled its requirement will conflict with that of the composition, canceling the composition.

说明：虽然代理 Command 的需求资源不会传递给组合，但它自身仍然需要对应的 subsystem。因此，如果需要同一 subsystem 的 Command 中有一个使用代理，那么它们都必须使用代理。否则，在调度代理 Command 时，它的需求资源会与组合的需求资源冲突，从而取消该组合。

```
// composition requirements are indexer and shooter, intake still reserved during its command but not afterwards
Commands.sequence(
    intake.intakeGamePiece().asProxy(), // we want to let the intake intake another game piece while we are processing this one
    indexer.processGamePiece(),
    shooter.aimAndShoot()
);
```

For cases that don't need to track the proxied command, [ScheduleCommand \(Java\)](#) schedules a specified command and ends instantly.

对于不需要跟踪代理 Command 的情况，[ScheduleCommand \(Java\)](#) 会调度指定的 Command，然后立即结束。

```
// ScheduleCommand ends immediately, so the sequence continues
new ScheduleCommand(Commands.waitSeconds(5.0))
    .andThen(Commands.print("This will be printed immediately!"))
```

Subclassing Compositions / 继承组合类

Command compositions can also be written as a constructor-only subclass of the most exterior composition type, passing the composition members to the superclass constructor. Consider the following from the Hatch Bot example project ([Java](#)):

还可以将 Command 组合写成最外层组合类型的子类：该子类只包含构造函数，并将组成 Command 传给父类构造函数。请看 Hatch Bot 示例项目（Java）中的做法：

```
package edu.wpi.first.wpilibj.examples.hatchbottraditional.commands;
import edu.wpi.first.wpilibj.examples.hatchbottraditional.Constants.AutoConstants;
import edu.wpi.first.wpilibj.examples.hatchbottraditional.subsystems.DriveSubsystem;
import edu.wpi.first.wpilibj.examples.hatchbottraditional.subsystems.HatchSubsystem;
import edu.wpi.first.wpilibj2.command.SequentialCommandGroup;
/** A complex auto command that drives forward, releases a hatch, and then drives backward. */
public class ComplexAuto extends SequentialCommandGroup {
    /**
     * Creates a new ComplexAuto.
     *
     * @param drive The drive subsystem this command will run on
     * @param hatch The hatch subsystem this command will run on
     */
    public ComplexAuto(DriveSubsystem drive, HatchSubsystem hatch) {
        addCommands(
            // Drive forward the specified distance
            new DriveDistance(
                AutoConstants.kAutoDriveDistanceInches, AutoConstants.kAutoDriveSpeed, drive),
            // Release the hatch
            new ReleaseHatch(hatch),
            // Drive backward the specified distance
            new DriveDistance(
                AutoConstants.kAutoBackupDistanceInches, -AutoConstants.kAutoDriveSpeed, drive));
    }
}
```

The advantages and disadvantages of this subclassing approach in comparison to others are discussed in [Subclassing Command Groups](#).

与其他方式相比，这种继承方式的优缺点在 Subclassing Command Groups 中讨论。

04 Subsystems / Subsystem

Subsystems are the basic unit of robot organization in the command-based paradigm. A subsystem is an abstraction for a collection of robot hardware that operates together as a unit. Subsystems form an [encapsulation](#) for this hardware, “hiding” it from the rest of the robot code and restricting access to it except through the subsystem’s public methods. Restricting the access in this way provides a single convenient place for code that might otherwise be duplicated in multiple places (such as scaling motor outputs or checking limit switches) if the subsystem internals were exposed. It also allows changes to the specific details of how the subsystem works (the “implementation”) to be isolated from the rest of robot code, making it far easier to make substantial changes if/when the design constraints change.

Subsystem 是 Command-Based 范式中组织机器人代码的基本单位。一个 subsystem 抽象了一组作为整体协同工作的机器人硬件。Subsystem 对这些硬件进行封装，向机器人的其余代码“隐藏”它们，并限制只能通过 subsystem 的公共方法访问它们。这样限制访问，可以把原本可能在多处重复的代码（例如调整电机输出比例或检查限位开关）集中在一个方便的位置。它还能将 subsystem 工作方式的具体细节（即“实现”）与机器人的其余代码隔离，使设计约束发生变化时更容易进行较大调整。

Subsystems also serve as the backbone of the `CommandScheduler`’s resource management system. Commands may declare resource requirements by specifying which subsystems they interact with; the scheduler will never concurrently schedule more than one command that requires a given subsystem. An attempt to schedule a command that requires a subsystem that is already-in-use will either interrupt the currently-running command or be ignored, based on the running command’s [Interruption Behavior](#).

Subsystem 也是 `CommandScheduler` 资源管理系统的基础。Command 可以通过指定自己与哪些 subsystem 交互来声明资源需求；调度器不会并发调度多个需要同一 subsystem 的 Command。若试图调度一个需要已被使用的 subsystem 的 Command，调度器会根据当前运行 Command 的中断行为，打断该 Command 或忽略新的调度请求。

Subsystems can be associated with “default commands” that will be automatically scheduled when no other command is currently using the subsystem. This is useful for “background” actions such as controlling the robot drive, keeping an arm held at a setpoint, or stopping motors when the subsystem isn’t used. Similar functionality can be achieved in the subsystem’s `periodic()` method, which is run once per run of the scheduler; teams should try to be consistent within their codebase about which functionality is achieved through either of these methods. Subsystems are represented in the command-based library by the [Subsystem](#) interface ([Java](#)).

可以为 subsystem 关联“默认 Command”，当没有其他 Command 正在使用该 subsystem 时，它会自动被调度。这适用于控制机器人底盘、将机械臂保持在 setpoint，或在 subsystem 未被使用时停止电机等“后台”动作。类似功能也可以在 subsystem 的 `periodic()` 方法中实现；调度器每运行一次就会调用该方法一次。队伍应尽量在自己的代码库中保持一致，明确哪些功能通过这两种方式中的哪一种实现。在 Command-Based 库中，subsystem 由 [Subsystem](#) 接口表示（Java）。

Creating a Subsystem / 创建 Subsystem

The recommended method to create a subsystem for most users is to subclass the abstract `SubsystemBase` class ([Java](#)), as seen in the command-based template ([Java](#)). The following example demonstrates how to create a simple subsystem in Java:

对大多数用户来说，推荐通过继承抽象类 `SubsystemBase` 创建 subsystem，如 Command-Based 模板（Java）所示。以下示例展示如何用 Java 创建一个简单的 subsystem：

```
import edu.wpi.first.wpilibj2.command.Command;
import edu.wpi.first.wpilibj2.command.SubsystemBase;
public class ExampleSubsystem extends SubsystemBase {
    /** Creates a new ExampleSubsystem. */
    public ExampleSubsystem() {}
    /**
     * Example command factory method.
     *
     * @return a command
     */
    public Command exampleMethodCommand() {
        // Inline construction of command goes here.
        // Subsystem::RunOnce implicitly requires `this` subsystem.
        return runOnce(
            () -> {
                /* one-time action goes here */
            });
    }
    /**
     * An example method querying a boolean state of the subsystem (for example, a digital sensor).
     *
     * @return value of some boolean subsystem state, such as a digital sensor.
     */
    public boolean exampleCondition() {
        // Query some boolean state, such as a digital sensor.
        return false;
    }
    @Override
    public void periodic() {
        // This method will be called once per scheduler run
    }
    @Override
    public void simulationPeriodic() {
        // This method will be called once per scheduler run during simulation
    }
}
```

This class contains a few convenience features on top of the basic `Subsystem` interface: it automatically calls the `register()` method in its constructor to register the subsystem with the scheduler (this is necessary for the `periodic()` method to be called when the scheduler runs), and also implements the `Sendable` interface so that it can be sent to the dashboard to display/log relevant status information.

除基本 `Subsystem` 接口外，这个类还提供了一些便利功能：它会在构造函数中自动调用 `register()` 方法，将 subsystem 注册到调度器中（只有这样，调度器运行时才会调用 `periodic()` 方法）；它还实现了 `Sendable` 接口，因此可以将相关状态信息发送到仪表板进行显示或记录。

Advanced users seeking more flexibility may simply create a class that implements the `Subsystem` interface. 需要更高灵活性的高级用户，也可以直接创建实现 `Subsystem` 接口的类。

Simple Subsystem Example / 简单 Subsystem 示例

What might a functional subsystem look like in practice? Below is a simple pneumatically-actuated hatch mechanism from the HatchBotTraditional example project ([Java](#)):

实际可用的 subsystem 会是什么样？下面是 HatchBotTraditional 示例项目（Java）中的一个由气动装置驱动的简单舱门机构：

```
package edu.wpi.first.wpilibj.examples.hatchbottraditional.subsystems;
import static edu.wpi.first.wpilibj.DoubleSolenoid.Value.kForward;
import static edu.wpi.first.wpilibj.DoubleSolenoid.Value.kReverse;
import edu.wpi.first.util.sendable.SendableBuilder;
import edu.wpi.first.wpilibj.DoubleSolenoid;
import edu.wpi.first.wpilibj.PneumaticsModuleType;
import edu.wpi.first.wpilibj.examples.hatchbottraditional.Constants.HatchConstants;
import edu.wpi.first.wpilibj2.command.SubsystemBase;
/** A hatch mechanism actuated by a single {@link DoubleSolenoid}. */
public class HatchSubsystemBase extends SubsystemBase {
    private final DoubleSolenoid m_hatchSolenoid =
        new DoubleSolenoid(
```

```

        PneumaticsModuleType.CTREPCM,
        HatchConstants.kHatchSolenoidPorts[0],
        HatchConstants.kHatchSolenoidPorts[1]);
    /** Grabs the hatch. */
    public void grabHatch() {
        m_hatchSolenoid.set(kForward);
    }
    /** Releases the hatch. */
    public void releaseHatch() {
        m_hatchSolenoid.set(kReverse);
    }
    @Override
    public void initSendable(SendableBuilder builder) {
        super.initSendable(builder);
        // Publish the solenoid state to telemetry.
        builder.addBooleanProperty("extended", () -> m_hatchSolenoid.get() == kForward, null);
    }
}

```

Notice that the subsystem hides the presence of the `DoubleSolenoid` from outside code (it is declared `private`), and instead publicly exposes two higher-level, descriptive robot actions: `grabHatch()` and `releaseHatch()`. It is extremely important that “implementation details” such as the double solenoid be “hidden” in this manner; this ensures that code outside the subsystem will never cause the solenoid to be in an unexpected state. It also allows the user to change the implementation (for instance, a motor could be used instead of a pneumatic) without any of the code outside of the subsystem having to change with it.

注意，`subsystem` 对外部代码隐藏了 `DoubleSolenoid` 的存在（它被声明为 `private`），转而公开两个层次更高、能描述机器人动作的方法：`grabHatch()` 和 `releaseHatch()`。以这种方式“隐藏”双作用电磁阀之类的“实现细节”非常重要；这确保了 `subsystem` 外部的代码不会让电磁阀进入意外状态。这样还允许用户更改实现（例如改用电机而非气动装置），而无须同步修改 `subsystem` 之外的代码。

Alternatively, instead of writing `void` public methods that are called from commands, we can define the public methods as factories that return a command. Consider the following from the `HatchBotInlined` example project ([Java](#)):

另一种做法是，不编写供 `Command` 调用的公共 `void` 方法，而是将公共方法定义为返回 `Command` 的工厂方法。请看 `HatchBotInlined` 示例项目（Java）中的实现：

```

package edu.wpi.first.wpilibj.examples.hatchbotinlined.subsystems;
import static edu.wpi.first.wpilibj.DoubleSolenoid.Value.kForward;
import static edu.wpi.first.wpilibj.DoubleSolenoid.Value.kReverse;
import edu.wpi.first.util.sendable.SendableBuilder;
import edu.wpi.first.wpilibj.DoubleSolenoid;
import edu.wpi.first.wpilibj.PneumaticsModuleType;
import edu.wpi.first.wpilibj.examples.hatchbotinlined.Constants.HatchConstants;
import edu.wpi.first.wpilibj2.command.Command;
import edu.wpi.first.wpilibj2.command.SubsystemBase;
/** A hatch mechanism actuated by a single {@link edu.wpi.first.wpilibj.DoubleSolenoid}. */
public class HatchSubsystem extends SubsystemBase {
    private final DoubleSolenoid m_hatchSolenoid =
        new DoubleSolenoid(
            PneumaticsModuleType.CTREPCM,
            HatchConstants.kHatchSolenoidPorts[0],
            HatchConstants.kHatchSolenoidPorts[1]);
    /** Grabs the hatch. */
    public Command grabHatchCommand() {
        // implicitly require `this`
        return this.runOnce(() -> m_hatchSolenoid.set(kForward));
    }
    /** Releases the hatch. */
    public Command releaseHatchCommand() {
        // implicitly require `this`
        return this.runOnce(() -> m_hatchSolenoid.set(kReverse));
    }
    @Override
    public void initSendable(SendableBuilder builder) {
        super.initSendable(builder);
        // Publish the solenoid state to telemetry.
        builder.addBooleanProperty("extended", () -> m_hatchSolenoid.get() == kForward, null);
    }
}

```


Note the qualification of the `runOnce` factory used here: this isn't the static factory in [Commands](#)! Subsystems have similar instance factories that return commands requiring `this` (Java) subsystem. Here, the `Subsystem.runOnce(Runnable)` factory ([Java](#)) is used.

注意这里使用的 `runOnce` 工厂方法的限定来源：它并不是 [Commands](#) 中的静态工厂方法！Subsystem 也提供类似的实例工厂方法，返回的 Command 会将 `this` subsystem 作为需求。这里使用的是 `Subsystem.runOnce(Runnable)` 工厂方法 (Java)。

For a comparison between these options, see [Instance Command Factory Methods](#).

有关这些方式的比较，请参阅 [Instance Command Factory Methods](#)。

Periodic / periodic

Subsystems have a `periodic` method that is called once every scheduler iteration (usually, once every 20 ms). This method is typically used for telemetry and other periodic actions that do not interfere with whatever command is requiring the subsystem.

Subsystem 有一个 `periodic` 方法，每个调度器周期都会调用一次（通常每 20 ms 一次）。此方法通常用于遥测，以及不会干扰当前占用该 subsystem 的 Command 的其他周期性动作。

```
@Override
public void periodic() {
    // Update the odometry in the periodic block
    m_odometry.update(
        Rotation2d.fromDegrees(getHeading()),
        m_leftEncoder.getDistance(),
        m_rightEncoder.getDistance());
    m_fieldSim.setRobotPose(getPose());
}
```

There is also a `simulationPeriodic()` method that is similar to `periodic()` except that it is only run during [Simulation](#) and can be used to update the state of the robot.

还有一个 `simulationPeriodic()` 方法，它与 `periodic()` 类似，但只在仿真期间运行，可用于更新机器人的状态。

Default Commands / 默认 Command

“Default commands” are commands that run automatically whenever a subsystem is not being used by another command. This can be useful for “background” actions such as controlling the robot drive, or keeping an arm held at a setpoint.

“默认 Command” 是在某个 subsystem 未被其他 Command 使用时自动运行的 Command。这可用于控制机器人底盘或将机械臂保持在 setpoint 等“后台”动作。

Setting a default command for a subsystem is very easy; one simply calls `CommandScheduler.getInstance().setDefaultCommand()`, or, more simply, the `setDefaultCommand()` method of the `Subsystem` interface:

为 subsystem 设置默认 Command 很简单：调用 `CommandScheduler.getInstance().setDefaultCommand()`，或者更简单地调用 `Subsystem` 接口的 `setDefaultCommand()` 方法：

```
CommandScheduler.getInstance().setDefaultCommand(exampleSubsystem, exampleCommand);
```

```
exampleSubsystem.setDefaultCommand(exampleCommand);
```

NOTE: A command that is assigned as the default command for a subsystem must require that subsystem.

说明：指定为某个 subsystem 的默认 Command，必须将该 subsystem 声明为需求。

05 Binding Commands to Triggers / 将 Command 绑定到 Trigger

Apart from autonomous commands, which are scheduled at the start of the autonomous period, and default commands, which are automatically scheduled whenever their subsystem is not currently in-use, the most common way to run a command is by binding it to a triggering event, such as a button being pressed by a human operator. The command-based paradigm makes this extremely easy to do.

除了在自动阶段开始时调度的自动 Command，以及在其 subsystem 未被使用时自动调度的默认 Command，运行 Command 最常见的方式是将其绑定到触发事件，例如操作员按下按钮。Command-Based 范式让这种绑定非常容易实现。

As mentioned earlier, command-based is a [declarative programming](#) paradigm. Accordingly, binding buttons to commands is done declaratively; the association of a button and a command is “declared” once, during robot initialization. The library then does all the hard work of checking the button state and scheduling (or canceling) the command as needed, behind-the-scenes. Users only need to worry about designing their desired UI setup - not about implementing it!

如前所述，Command-Based 是一种声明式编程范式。因此，按钮与 Command 的绑定也是声明式的：只需在机器人初始化期间“声明”一次按钮与 Command 的关联。随后，库会在后台完成检查按钮状态并按需调度（或取消）Command 的工作。用户只需设计期望的操作界面，而不必自行实现这些底层逻辑。

Command binding is done through the [Trigger](#) class ([Java](#)).

Command 绑定通过 [Trigger](#) 类完成 (Java)。

Getting a Trigger Instance / 获取 Trigger 实例

To bind commands to conditions, we need a [Trigger](#) object. There are three ways to get a Trigger object: 要将 Command 绑定到条件，需要一个 [Trigger](#) 对象。获取 [Trigger](#) 对象有三种方式：

HID Factories / HID 工厂方法

The command-based HID classes contain factory methods returning a [Trigger](#) for a given button. [CommandGenericHID](#) has an index-based [button\(int\)](#) factory ([Java](#)), and its subclasses [CommandXboxController](#) ([Java](#)), [CommandPS4Controller](#) ([Java](#)), and [CommandJoystick](#) ([Java](#)) have named factory methods for each button. Command-Based HID 类提供了工厂方法，可为指定按钮返回 [Trigger](#)。[CommandGenericHID](#) 有按索引查找的 [button\(int\)](#) 工厂方法；其子类 [CommandXboxController](#)、[CommandPS4Controller](#) 和 [CommandJoystick](#) 则为各按钮提供了按名称调用的工厂方法。

```
CommandXboxController exampleCommandController = new CommandXboxController(1); // Creates a CommandXboxController on port 1.
Trigger xButton = exampleCommandController.x(); // Creates a new Trigger object for the `X` button on exampleCommandController
```

JoystickButton / JoystickButton

Alternatively, the [regular HID classes](#) can be used and passed to create an instance of [JoystickButton](#) ([Java](#)), a constructor-only subclass of [Trigger](#):

也可以使用普通 HID 类，并将其传入以创建 [JoystickButton](#) 实例。[JoystickButton](#) 是 [Trigger](#) 的一个只有构造函数的子类：

```
XboxController exampleController = new XboxController(2); // Creates an XboxController on port 2.
Trigger yButton = new JoystickButton(exampleController, XboxController.Button.kY.value); // Creates a new JoystickButton object for the 'Y' button on exampleController
```

Arbitrary Triggers / 任意条件的 Trigger

While binding to HID buttons is by far the most common use case, users may want to bind commands to arbitrary triggering events. This can be done inline by passing a lambda to the constructor of `Trigger`:

虽然绑定 HID 按钮是最常见的用法，用户也可能希望把 Command 绑定到任意触发事件。可以直接向 `Trigger` 构造函数传入 lambda：

```
DigitalInput limitSwitch = new DigitalInput(3); // Limit switch on DIO 3
Trigger exampleTrigger = new Trigger(limitSwitch::get);
```

Trigger Bindings / Trigger 绑定方式

There are a number of bindings available for the `Trigger` class. All of these bindings will automatically schedule a command when a certain trigger activation event occurs - however, each binding has different specific behavior.

`Trigger` 类提供了多种绑定方式。它们都会在特定的触发器激活事件发生时自动调度 Command，但各自的具体行为不同。

`Trigger` objects do not need to survive past the call to a binding method, so the binding methods may be simply called on a temp. Remember that button binding is declarative: bindings only need to be declared once, ideally some time during robot initialization. The library handles everything else.

`Trigger` 对象在调用绑定方法后无须继续存活，因此可以直接对临时对象调用绑定方法。请记住，按钮绑定是声明式的：只需声明一次绑定，最好在机器人初始化期间完成。其余工作由库处理。

NOTE: The `Button` subclass is deprecated, and usage of its binding methods should be replaced according to the respective deprecation messages in the API docs.

说明：`Button` 子类已经弃用，应按照 API 文档中的相应弃用说明替换对其绑定方法的使用。

onTrue / onTrue

This binding schedules a command when a trigger changes from `false` to `true` (or, accordingly, when a button changes is initially pressed). The command will be scheduled on the iteration when the state changes, and will not be scheduled again unless the trigger becomes `false` and then `true` again (or the button is released and then re-pressed).

当触发器从 `false` 变为 `true`（即按钮刚被按下）时，此绑定会调度一个 Command。Command 在状态变化的那个周期被调度；除非触发器先变为 `false`、再变为 `true`（即松开按钮后再次按下），否则不会再次调度。

```
// Retract the intake with the Y button
m_driverController.y().onTrue(m_intake.retractCommand());
```

The `onFalse` binding is identical, only that it schedules on `false` instead of on `true`.

`onFalse` 绑定的行为相同，只是它在状态变为 `false` 时调度，而不是在变为 `true` 时调度。

whileTrue / whileTrue

This binding schedules a command when a trigger changes from `false` to `true` (or, accordingly, when a button is initially pressed) and cancels it when the trigger becomes `false` again (or the button is released). The command will not be re-scheduled if it finishes while the trigger is still `true`. For the command to restart if it finishes while the trigger is `true`, wrap the command in a `RepeatCommand`, or use a `RunCommand` instead of an `InstantCommand`.

当触发器从 `false` 变为 `true`（即按钮刚被按下）时，此绑定会调度一个 `Command`；当触发器再次变为 `false`（即按钮松开）时会取消它。如果 `Command` 在触发器仍为 `true` 时结束，不会重新调度。若希望它在触发器仍为 `true` 时结束后重新启动，可以用 `RepeatCommand` 包装该 `Command`，或者使用 `RunCommand` 代替 `InstantCommand`。

```
// Schedule `exampleMethodCommand` when the Xbox controller's B button is pressed,
// cancelling on release.
m_driverController.b().whileTrue(m_exampleSubsystem.exampleMethodCommand());
```

The `whileFalse` binding is identical, only that it schedules on `false` and cancels on `true`.

`whileFalse` 绑定的行为相同，只是它在 `false` 时调度，在 `true` 时取消。

toggleOnTrue / toggleOnTrue

This binding toggles a command, scheduling it when a trigger changes from `false` to `true` (or a button is initially pressed), and canceling it under the same condition if the command is currently running. Note that while this functionality is supported, toggles are not a highly-recommended option for user control, as they require the driver to keep track of the robot state. The preferred method is to use two buttons; one to turn on and another to turn off. Using a [StartEndCommand](#) or a [ConditionalCommand](#) is a good way to specify the commands that you want to be toggled between.

此绑定会切换 `Command` 状态：当触发器从 `false` 变为 `true`（即按钮刚被按下）时调度它；如果该 `Command` 已在运行，则在同样条件下取消它。虽然支持这一功能，但切换开关并不是特别推荐的用户控制方式，因为驾驶员必须持续记住机器人当前状态。更推荐使用两个按钮：一个用于开启，另一个用于关闭。使用 `StartEndCommand` 或 `ConditionalCommand`，可以方便地指定要在其间切换的 `Command`。

```
// Toggle compressor with the Start button
m_driverController.start().toggleOnTrue(m_pneumatics.disableCompressorCommand());
```

The `toggleOnFalse` binding is identical, only that it toggles on `false` instead of on `true`.

`toggleOnFalse` 绑定的行为相同，只是它在变为 `false` 时切换，而不是在变为 `true` 时切换。

Chaining Calls / 链式调用

It is useful to note that the command binding methods all return the trigger that they were called on, and thus can be chained to bind multiple commands to different states of the same trigger. For example:

需要注意，所有 `Command` 绑定方法都会返回调用该方法的触发器，因此可以链式调用，为同一触发器的不同状态绑定多个 `Command`。例如：

```
exampleButton
    // Binds a FooCommand to be scheduled when the button is pressed
    .onTrue(new FooCommand())
    // Binds a BarCommand to be scheduled when that same button is released
    .onFalse(new BarCommand());
```

Composing Triggers / 组合 Trigger

The [Trigger](#) class can be composed to create composite triggers through the [and\(\)](#), [or\(\)](#), and [negate\(\)](#) methods. For example:

可以通过 [and\(\)](#)、[or\(\)](#) 和 [negate\(\)](#) 方法组合 [Trigger](#) 对象，创建复合触发器。例如：

```
// Binds an ExampleCommand to be scheduled when both the 'X' and 'Y' buttons of the driver gamepad are pressed
exampleCommandController.x()
    .and(exampleCommandController.y())
    .onTrue(new ExampleCommand());
```

Debouncing Triggers / Trigger 去抖

To avoid rapid repeated activation, triggers (especially those originating from digital inputs) can be debounced with the [WPILib Debouncer class](#) using the [debounce](#) method:

为避免快速重复激活，可以使用 [debounce](#) 方法和 [WPILib Debouncer](#) 类对触发器（尤其是来自数字输入的触发器）进行去抖：

```
// debounces exampleButton with a 0.1s debounce time, rising edges only
exampleButton.debounce(0.1).onTrue(new ExampleCommand());
// debounces exampleButton with a 0.1s debounce time, both rising and falling edges
exampleButton.debounce(0.1, Debouncer.DebounceType.kBoth).onTrue(new ExampleCommand());
```

06 Structuring a Command-Based Robot Project / Command-Based 机器人项目的结构

While users are free to use the command-based libraries however they like (and advanced users are encouraged to do so), new users may want some guidance on how to structure a basic command-based robot project.

用户可以按自己希望的方式使用 Command-Based 库（也鼓励高级用户这样做），但新用户可能需要一些指导，了解如何组织基本的 Command-Based 机器人项目。

A standard template for a command-based robot project is included in the WPILib examples repository ([Java](#)). This section will walk users through the structure of this template.

WPILib 示例仓库中包含一个标准的 Command-Based 机器人项目模板（Java）。本节将介绍该模板的结构。

The root package/directory generally will contain four classes:

根 package / 目录通常包含四个类：

`Main`, which is the main robot application (Java only). New users should not touch this class. `Robot`, which is responsible for the main control flow of the robot code. `RobotContainer`, which holds robot subsystems and commands, and is where most of the declarative robot setup (e.g. button bindings) is performed. `Constants`, which holds globally-accessible constants to be used throughout the robot.

`Main` 是机器人应用程序的入口（仅 Java）。新用户不应修改这个类。`Robot` 负责机器人代码的主要控制流程。`RobotContainer` 保存机器人的 subsystem 和 Command，大部分声明式机器人设置（例如按钮绑定）都在其中完成。`Constants` 保存整个机器人代码都可访问的常量。

The root directory will also contain two sub-packages/sub-directories: `Subsystems` contains all user-defined subsystem classes. `Commands` contains all user-defined command classes.

根目录还包含两个子 package / 子目录：`Subsystems` 包含所有用户定义的 subsystem 类；`Commands` 包含所有用户定义的 Command 类。

Robot / Robot

As `Robot` ([Java](#)) is responsible for the program's control flow, and command-based is an declarative paradigm designed to minimize the amount of attention the user has to pay to explicit program control flow, the `Robot` class of a command-based project should be mostly empty. However, there are a few important things that must be included

由于 `Robot` (Java) 负责程序的控制流程，而 Command-Based 是一种旨在减少用户对显式程序控制流程关注的声明式范式，因此 Command-Based 项目中的 `Robot` 类应基本保持空白。不过，仍有几项重要内容必须包含其中。

```
/**
 * This function is run when the robot is first started up and should be used for any
 * initialization code.
 */
public Robot() {
    // Instantiate our RobotContainer. This will perform all our button bindings, and put our
    // autonomous chooser on the dashboard.
    m_robotContainer = new RobotContainer();
}
```


In Java, an instance of `RobotContainer` should be constructed during the `Robot` constructor - this is important, as most of the declarative robot setup will be called from the `RobotContainer` constructor.

在 Java 中，应在 `Robot` 构造函数内构造 `RobotContainer` 实例。这很重要，因为大部分声明式机器人设置都将从 `RobotContainer` 构造函数中执行。

```
/**
 * This function is called every 20 ms, no matter the mode. Use this for items like diagnostics
 * that you want ran during disabled, autonomous, teleoperated and test.
 *
 * <p>This runs after the mode specific periodic functions, but before LiveWindow and
 * SmartDashboard integrated updating.
 */
@Override
public void robotPeriodic() {
    // Runs the Scheduler. This is responsible for polling buttons, adding newly-scheduled
    // commands, running already-scheduled commands, removing finished or interrupted commands,
    // and running subsystem periodic() methods. This must be called from the robot's periodic
    // block in order for anything in the Command-based framework to work.
    CommandScheduler.getInstance().run();
}
```

The inclusion of the `CommandScheduler.getInstance().run()` call in the `robotPeriodic()` method is essential; without this call, the scheduler will not execute any scheduled commands. Since `TimedRobot` runs with a default main loop frequency of 50Hz, this is the frequency with which periodic command and subsystem methods will be called. It is not recommended for new users to call this method from anywhere else in their code.

在 `robotPeriodic()` 方法中调用 `CommandScheduler.getInstance().run()` 至关重要；缺少这次调用，调度器就不会执行任何已调度的 `Command`。由于 `TimedRobot` 的主循环默认频率是 50Hz，`Command` 和 `Subsystem` 的 `periodic` 方法也会以这一频率被调用。不建议新用户从代码中的其他位置调用该方法。

```
/** This autonomous runs the autonomous command selected by your {@link RobotContainer} class. */
@Override
public void autonomousInit() {
    m_autonomousCommand = m_robotContainer.getAutonomousCommand();
    // schedule the autonomous command (example)
    if (m_autonomousCommand != null) {
        CommandScheduler.getInstance().schedule(m_autonomousCommand);
    }
}
```

The `autonomousInit()` method schedules an autonomous command returned by the `RobotContainer` instance. The logic for selecting which autonomous command to run can be handled inside of `RobotContainer`.

`autonomousInit()` 方法会调度由 `RobotContainer` 实例返回的自动 `Command`。选择运行哪个自动 `Command` 的逻辑可以放在 `RobotContainer` 中处理。

```
@Override
public void teleopInit() {
    // This makes sure that the autonomous stops running when
    // teleop starts running. If you want the autonomous to
    // continue until interrupted by another command, remove
    // this line or comment it out.
    if (m_autonomousCommand != null) {
        m_autonomousCommand.cancel();
    }
}
```

The `teleopInit()` method cancels any still-running autonomous commands. This is generally good practice.

`teleopInit()` 方法会取消任何仍在运行的自动 `Command`。通常这是良好的实践。

Advanced users are free to add additional code to the various init and periodic methods as they see fit; however, it should be noted that including large amounts of imperative robot code in `Robot.java` is contrary to the declarative design philosophy of the command-based paradigm, and can result in confusingly-structured/disorganized code.

高级用户可以按需在各个 `init` 和 `periodic` 方法中添加代码；但需要注意，如果在 `Robot.java` 中加入大量命令式机器人代码，就会背离 Command-Based 范式的声明式设计理念，还可能导致代码结构混乱、难以组织。

RobotContainer / RobotContainer

This class ([Java](#)) is where most of the setup for your command-based robot will take place. In this class, you will define your robot's subsystems and commands, bind those commands to triggering events (such as buttons), and specify which command you will run in your autonomous routine. There are a few aspects of this class new users may want explanations for:

这个类 (Java) 是 Command-Based 机器人的大部分设置工作发生的地方。在其中定义机器人的 subsystem 和 Command，将这些 Command 绑定到触发事件（例如按钮），并指定自动程序中运行的 Command。新用户可能需要了解这个类的几个方面：

```
private final ExampleSubsystem m_exampleSubsystem = new ExampleSubsystem();
```

Notice that subsystems are declared as private fields in `RobotContainer`. This is in stark contrast to the previous incarnation of the command-based framework, but is much more-aligned with agreed-upon object-oriented best-practices. If subsystems are declared as global variables, it allows the user to access them from anywhere in the code. While this can make certain things easier (for example, there would be no need to pass subsystems to commands in order for those commands to access them), it makes the control flow of the program much harder to keep track of as it is not immediately obvious which parts of the code can change or be changed by which other parts of the code. This also circumvents the ability of the resource-management system to do its job, as ease-of-access makes it easy for users to accidentally make conflicting calls to subsystem methods outside of the resource-managed commands.

注意，subsystem 在 `RobotContainer` 中声明为私有字段。这与上一代 Command-Based 框架的做法形成鲜明对比，却更符合公认的面向对象最佳实践。如果将 subsystem 声明为全局变量，就可以从代码的任意位置访问它们。虽然这会让某些事情更容易（例如不必把 subsystem 传给 Command，Command 也能访问它们），但也会使程序控制流程更难追踪，因为哪些代码能够更改哪些其他代码、又会被哪些代码更改，并不能一眼看出。这也会绕过资源管理系统的作用：过于方便的访问，使用户容易在受资源管理的 Command 之外意外地对 subsystem 方法发出冲突调用。

```
return Autos.exampleAuto(m_exampleSubsystem);
```

Since subsystems are declared as private members, they must be explicitly passed to commands (a pattern called “dependency injection”) in order for those commands to call methods on them. This is done here with `ExampleCommand`, which is passed an `ExampleSubsystem` instance.

由于 subsystem 被声明为私有成员，必须显式地将它们传给 Command（这一模式称为“依赖注入”），Command 才能调用其方法。这里通过将 `ExampleSubsystem` 实例传给 `ExampleCommand` 来实现。

```
/**
 * Use this method to define your trigger->command mappings. Triggers can be created via the
 * {@link Trigger#Trigger(java.util.function.BooleanSupplier)} constructor with an arbitrary
 * predicate, or via the named factories in {@link
 * edu.wpi.first.wpilibj2.command.button.CommandGenericHID}'s subclasses for {@link
 * CommandXboxController Xbox}/{@link edu.wpi.first.wpilibj2.command.button.CommandPS4Controller
 * PS4} controllers or {@link edu.wpi.first.wpilibj2.command.button.CommandJoystick Flight
 * joysticks}.
 */
private void configureBindings() {
    // Schedule `ExampleCommand` when `exampleCondition` changes to `true`
    new Trigger(m_exampleSubsystem::exampleCondition)
        .onTrue(new ExampleCommand(m_exampleSubsystem));
    // Schedule `exampleMethodCommand` when the Xbox controller's B button is pressed,
    // cancelling on release.
    m_driverController.b().whileTrue(m_exampleSubsystem.exampleMethodCommand());
}
```

```
}
```

As mentioned before, the `RobotContainer()` constructor is where most of the declarative setup for the robot should take place, including button bindings, configuring autonomous selectors, etc. If the constructor gets too “busy,” users are encouraged to migrate code into separate subroutines (such as the `configureBindings()` method included by default) which are called from the constructor.

如前所述，`RobotContainer()` 构造函数是进行大部分声明式机器人设置的地方，包括按钮绑定、自动程序选择器配置等。如果构造函数变得过于繁杂，建议将代码移到由构造函数调用的独立方法中，例如默认提供的 `configureBindings()` 方法。

```
/**
 * Use this to pass the autonomous command to the main {@link Robot} class.
 *
 * @return the command to run in autonomous
 */
public Command getAutonomousCommand() {
    // An example command will be run in autonomous
    return Autos.exampleAuto(m_exampleSubsystem);
}
```

Finally, the `getAutonomousCommand()` method provides a convenient way for users to send their selected autonomous command to the main `Robot` class (which needs access to it to schedule it when autonomous starts).

最后，`getAutonomousCommand()` 方法提供了一种方便的方式，将用户选定的自动 `Command` 传给主 `Robot` 类；主类需要获取它，才能在自动阶段开始时调度。

Constants / Constants

The `Constants` class ([Java](#)) is where globally-accessible robot constants (such as speeds, unit conversion factors, PID gains, and sensor/motor ports) can be stored. It is recommended that users separate these constants into individual inner classes corresponding to subsystems or robot modes, to keep variable names shorter.

`Constants` 类 (Java) 可以存放全局可访问的机器人常量，例如速度、单位换算系数、PID 增益，以及传感器和电机端口。建议按 `subsystem` 或机器人模式，将这些常量分到不同的内部类中，以缩短变量名称。

In Java, all constants should be declared `public static final` so that they are globally accessible and cannot be changed.

在 Java 中，所有常量都应声明为 `public static final`，这样它们便可被全局访问，也不能再被重新赋值。

For more illustrative examples of what a `constants` class should look like in practice, see those of the various command-based example projects:

要查看 `constants` 类在实践中应是什么样，可以参考各个 Command-Based 示例项目：

- Hatchbot ([Java](#))

Hatchbot (Java)

- RapidReactCommandBot ([Java](#))

RapidReactCommandBot (Java)

In Java, it is recommended that the constants be used from other classes by statically importing the necessary inner class. An `import static` statement imports the static namespace of a class into the class in which you are working, so that any `static` constants can be referenced directly as if they had been defined in that class:

在 Java 中，建议通过静态导入所需的内部类，在其他类中使用这些常量。`import static` 语句会将一个类的静态命名空间导入当前类，使任何 `static` 常量都能像定义在当前类中一样被直接引用：

```
import static edu.wpi.first.wpilibj.templates.commandbased.Constants.OIConstants.*;
```

Subsystems / Subsystem

User-defined subsystems should go in this package/directory.

用户定义的 subsystem 应放在此 package / 目录中。

Commands / Command

User-defined commands should go in this package/directory.

用户定义的 Command 应放在此 package / 目录中。

07 Organizing Command-Based Robot Projects / 组织 Command-Based 机器人项目

As robot code becomes more complicated, navigating, understanding, and maintaining the code takes up more and more time and energy. Making changes to the code often becomes more difficult, sometimes for reasons that have very little to do with the actual complexity of the underlying logic. For a simplified example: putting the logic for many unrelated robot functions into a single 1000-line file makes it difficult to find a specific piece of code within that file, particularly under stress at a competition. But spreading out closely related logic across dozens of tiny files is often just as difficult to navigate.

随着机器人代码越来越复杂，浏览、理解和维护代码会占用越来越多的时间和精力。修改代码也往往越来越困难，有时原因与底层逻辑本身的复杂度关系很小。举个简化的例子：将许多互不相关的机器人功能逻辑放入同一个 1000 行文件，会让人很难在该文件中找到某段代码，尤其是在比赛压力下。但把紧密相关的逻辑分散到几十个小文件中，往往同样难以浏览。

This is not a problem unique to FRC, and in fact, good organization only becomes more and more critical as software projects become bigger and bigger. The “best” organization system is a perennial topic of debate, much like the “best” programming language, but in the end, the choice (in both cases) comes down to the specific task at hand and the programmer (or programmers) implementing said task. Even in the relatively small space of FRC robot programming, there is no right answer. The best choice for a given team will depend on the nature of the specific robot code, team structure, and pure personal preference.

这并非 FRC 独有的问题。事实上，软件项目越大，良好的组织就越重要。与“最好”的编程语言一样，“最好”的组织方式也是长期争论的话题。但归根结底，两者的选择都取决于具体任务，以及实现该任务的程序员。即使在范围相对较小的 FRC 机器人编程中，也没有唯一正确的答案。某支队伍的最佳选择取决于具体机器人代码的性质、队伍结构和个人偏好。

This article discusses various facets of command-based robot program design that advanced FRC programmers may want to be aware of when writing code. It is not a prescriptive tutorial, though it presents some recommended best practices. If this level of choice seems daunting, however, many teams have been highly successful while sticking closely to WPILib’s example code and guidelines. However, this discussion may be of interest to intermediate and advanced programmers who want to make their code not only effective, but flexible, easily changeable, and sometimes even beautiful.

本文讨论 Command-Based 机器人程序设计的多个方面，供编写代码的资深 FRC 程序员参考。它不是一篇规定必须如何做的教程，尽管也提出了一些推荐的最佳实践。如果这么多选择令人望而生畏，许多队伍严格遵循 WPILib 的示例代码和指南也取得了很大成功。不过，对希望代码不仅有效，还具备灵活性、易于修改，甚至具有美感的中高级程序员来说，以下讨论可能有帮助。

Why Care About Organization? / 为什么要关心代码组织？

Good code organization will rarely make or break a team’s competitive ability—but it does mean easier debugging, faster modifications, nicer-looking code, and happier programmers. While it’s impossible to define “good” organization by way of what the code looks like from the inside, it’s easier to define in terms of what the robot’s software looks like from the outside.

良好的代码组织很少直接决定队伍的比赛能力，但它意味着更容易调试、更快完成修改、更美观的代码，以及更满意的程序员。虽然很难根据代码内部的样子定义何为“良好”的组织，但可以更容易地从机器人软件在外部呈现出的效果来定义。

What Good Organization Looks Like / 良好的组织是什么样

When code is well-designed and well-organized, the code's internal structure is intuitive and easily comprehensible. Cumbersome boilerplate is minimized, meaning that new robot functionality can often be added with just a few lines of code. When a constant value (such as the speed of the robot's intake) needs to be changed, it only needs to change in one place. If multiple programmers are working together, they can easily understand each others' work. Bugs are rare, since it is difficult to accidentally introduce unintended behavior (such as creating a command that does not require necessary subsystems). Implementing more advanced functions like unit tests is easier, since the code is abstracted away from the physical hardware. Programmers are happy (most of the time).

当代码设计和组织良好时，其内部结构直观、容易理解。繁琐的样板代码被降到最低，新机器人功能往往只需几行代码就能加入。需要更改某个常量值（例如机器人吸取机构的速度）时，只需在一个位置修改。如果有多名程序员协作，他们可以轻松理解彼此的工作。错误较少，因为很难意外引入非预期行为（例如创建一个没有声明必要 subsystem 需求资源的 Command）。由于代码与实体硬件隔离，实现单元测试等高级功能也更加容易。程序员（大多数时候）也会感到满意。

What Bad Organization Looks Like / 糟糕的组织是什么样

Poorly organized code often has internal structure that makes little to no sense, even to whoever wrote it. When functionality has to be added or changed, it often breaks unrelated parts of the robot: adding automatic shooter control might introduce a bug in the climbing sequence for unclear reasons. Alternatively, the organizational framework might be so strict that it's impossible to implement necessary behavior, requiring nasty hacks or workarounds. Many lines of boilerplate code are needed for simple robot logic. Constants are scattered across the codebase, and changing basic behavior often requires making the same change to many different files. Collaboration among multiple programmers is difficult or impossible.

组织不佳的代码，其内部结构通常连作者自己也难以理解。添加或修改功能时，往往会破坏机器人中无关的部分：例如加入发射器自动控制，却莫名其妙地在攀爬序列中引入错误。另一种情况是，组织框架过于严格，无法实现必要行为，只能使用难看的变通写法。简单的机器人逻辑也需要许多行样板代码。常量散落在代码库各处，改变基本行为往往要在多个文件中重复修改。多名程序员也很难协作，甚至无法协作。

Defining Commands / 定义 Command

In larger robot codebases, multiple copies of the same command need to be used in many different places. For instance, a command that runs a robot's intake might be used in teleop, bound to a certain button; as part of a complicated command group for an autonomous routine; and as part of a self-test sequence.

在较大的机器人代码库中，同一种 Command 的多个实例需要在不同位置使用。例如，运行机器人吸取机构的 Command，可能在遥控阶段绑定到某个按钮，也可能成为复杂自动程序 Command 组的一部分，或成为自检序列的一部分。

As an example, let's look at some ways to define a simple command that simply runs the robot's intake forward at full power until canceled.

下面以一个简单 Command 为例，看看定义方式：它让机器人的吸取机构以全功率正转，直到被取消。

Inline Commands / 内联 Command

The easiest and most expressive way to do this is with a `StartEndCommand`:

最简单、表达力最强的方式是使用 `StartEndCommand`：

```
Command runIntake = Commands.startEnd(() -> intake.set(1.0), () -> intake.set(0.0), intake);
```


This is sufficient for commands that are only used once. However, for a command like this that might get used in many different autonomous routines and button bindings, inline commands everywhere means a lot of repetitive code:

对于只使用一次的 Command，这已经足够。但是，像这样可能在多个自动程序和按钮绑定中使用的 Command，如果每次都内联编写，就会产生大量重复代码：

```
// RobotContainer.java
intakeButton.whileTrue(Commands.startEnd(() -> intake.set(1.0), () -> intake.set(0.0), intake));
Command intakeAndShoot = Commands.startEnd(() -> intake.set(1.0), () -> intake.set(0.0), intake)
    .alongWith(new RunShooter(shooter));
Command autonomousCommand = Commands.sequence(
    Commands.startEnd(() -> intake.set(1.0), () -> intake.set(0.0), intake).withTimeout(5.0),
    Commands.waitSeconds(3.0),
    Commands.startEnd(() -> intake.set(1.0), () -> intake.set(0.0), intake).withTimeout(5.0)
);
```

Creating one `StartEndCommand` instance and putting it in a variable won't work here, since once an instance of a command is added to a command group it is effectively “owned” by that command group and cannot be used in any other context.

创建一个 `StartEndCommand` 实例并存入变量，在这里行不通；实例一旦加入 Command 组，实际上便由该组“拥有”，不能在其他上下文中使用。

Instance Command Factory Methods / 实例 Command 工厂方法

One way to solve this quandary is using the “factory method” design pattern: a function that returns a new object every invocation, according to some specification. Using [command composition](#), a factory method can construct a complex command object with merely a few lines of code.

解决这个问题的一种方式是使用“工厂方法”设计模式：一个按特定规则、每次调用都返回新对象的函数。借助 Command 组合，工厂方法只需几行代码就能构造复杂的 Command 对象。

For example, a command like the intake-running command is conceptually related to exactly one subsystem: the `Intake`. As such, it makes sense to put a `runIntakeCommand` method as an instance method of the `Intake` class:

例如，运行吸取机构的 Command 在概念上只与一个 subsystem 相关：`Intake`。因此，将 `runIntakeCommand` 设为 `Intake` 类的实例方法是合理的：

NOTE: In this document we will name factory methods as `lowerCamelCaseCommand`, but teams may decide on other conventions. In general, it is recommended to end the method name with `Command` if it might otherwise be confused with an ordinary method (e.g. `intake.run` might be the name of a method that simply turns on the intake).

说明：本文将工厂方法命名为

`lowerCamelCaseCommand`，但队伍也可以采用其他约定。一般来说，如果方法名可能与普通方法混淆，建议以 `Command` 结尾（例如，`intake.run` 也可能是一个直接启动吸取机构的方法名）。

```
public class Intake extends SubsystemBase {
    // [code for motor controllers, configuration, etc.]
    // ...
    public Command runIntakeCommand() {
        // implicitly requires `this`
        return this.startEnd(() -> this.set(1.0), () -> this.set(0.0));
    }
}
```

Notice how since we are in the `Intake` class, we no longer refer to `intake`; instead, we use the `this` keyword to refer to the current instance.

注意，因为此时处于 `Intake` 类中，不再通过 `intake` 引用它，而是使用 `this` 关键字引用当前实例。

Since we are inside the `Intake` class, technically we can access `private` variables and methods directly from within the `runIntakeCommand` method, thus not needing intermediary methods. (For example, the `runIntakeCommand` method can directly interface with the motor controller objects instead of calling `set()`.) On the other hand, these intermediary methods can reduce code duplication and increase encapsulation. Like many other choices outlined in this document, this tradeoff is a matter of personal preference on a case-by-case basis.

由于位于 `Intake` 类内部，从技术上讲，可以在 `runIntakeCommand` 方法中直接访问 `private` 变量和方法，无须经过中间方法。（例如，`runIntakeCommand` 可以直接与电机控制器对象交互，而不调用 `set()`。）另一方面，中间方法可以减少代码重复并提高封装程度。与本文介绍的许多选择一样，需要针对具体情况按个人偏好权衡。

Using this new factory method in command groups and button bindings is highly expressive:

在 Command 组和按钮绑定中使用这个新的工厂方法，表达非常清晰：

```
intakeButton.whileTrue(intake.runIntakeCommand());
Command intakeAndShoot = intake.runIntakeCommand().alongWith(new RunShooter(shooter));
Command autonomousCommand = Commands.sequence(
    intake.runIntakeCommand().withTimeout(5.0),
    Commands.waitSeconds(3.0),
    intake.runIntakeCommand().withTimeout(5.0)
);
```

Adding a parameter to the `runIntakeCommand` method to provide the exact percentage to run the intake is easy and allows for even more flexibility.

向 `runIntakeCommand` 方法添加一个参数，以指定吸取机构运行的具体百分比并不困难，也能提供更大的灵活性。

```
public Command runIntakeCommand(double percent) {
    return new StartEndCommand(() -> this.set(percent), () -> this.set(0.0), this);
}
```

For instance, this code creates a command group that runs the intake forwards for two seconds, waits for two seconds, and then runs the intake backwards for five seconds.

例如，以下代码创建一个 Command 组：先让吸取机构正转两秒，等待两秒，然后反转五秒。

```
Command intakeRunSequence = intake.runIntakeCommand(1.0).withTimeout(2.0)
    .andThen(Commands.waitSeconds(2.0))
    .andThen(intake.runIntakeCommand(-1.0).withTimeout(5.0));
```

This approach is recommended for commands that are conceptually related to only a single subsystem, and is very concise. However, it doesn't fare well with commands related to more than one subsystem: passing in other subsystem objects is unintuitive and can cause race conditions and circular dependencies, and thus should be avoided. Therefore, this approach is best suited for single-subsystem commands, and should be used only for those cases.

对于在概念上只与一个 subsystem 相关的 Command，推荐采用这一简洁方式。但它不适合涉及多个 subsystem 的 Command：传入其他 subsystem 对象并不直观，而且可能造成状态竞争和循环依赖，因此应避免。此方法最适用于单 subsystem Command，也应只在这种情况下使用。

Static Command Factories / 静态 Command 工厂

Instance factory methods work great for single-subsystem commands. However, complicated robot actions (like the ones often required during the autonomous period) typically need to coordinate multiple subsystems at once. When we want to define an inline command that uses multiple subsystems, it doesn't make sense for the command factory to live in any single one of those subsystems. Instead, it can be cleaner to define the command factory methods statically in some external class:

实例工厂方法非常适合单 subsystem Command。但复杂的机器人动作（例如自动阶段常见的动作）通常需要同时协调多个 subsystem。要定义使用多个 subsystem 的内联 Command，将工厂方法放在其中某一个 subsystem 中并不合理。将 Command 工厂方法静态地定义在某个外部类中可能更清晰：

NOTE: The `sequence` and `parallel` static factories construct sequential and parallel command groups: this is equivalent to the `andThen` and `alongWith` decorators, but can be more readable. Their use is a matter of personal preference.

说明：`sequence` 和 `parallel` 静态工厂方法分别构造顺序和并行 Command 组：这与 `andThen` 和 `alongWith` 装饰器等价，但可能更易读。选择哪一种取决于个人偏好。

```
public class AutoRoutines {
    public static Command driveAndIntake(Drivetrain drivetrain, Intake intake) {
        return Commands.sequence(
            Commands.parallel(
                drivetrain.driveCommand(0.5, 0.5),
                intake.runIntakeCommand(1.0)
            ).withTimeout(5.0),
            Commands.parallel(
                drivetrain.stopCommand();
                intake.stopCommand();
            )
        );
    }
}
```

Non-Static Command Factories / 非静态 Command 工厂

If we want to avoid the verbosity of adding required subsystems as parameters to our factory methods, we can instead construct an instance of our `AutoRoutines` class and inject our subsystems through the constructor:

如果希望避免在工厂方法中反复把所需 subsystem 作为参数传入，可以改为构造一个 `AutoRoutines` 类的实例，并通过构造函数注入 subsystem：

```
public class AutoRoutines {
    private Drivetrain drivetrain;
    private Intake intake;
    public AutoRoutines(Drivetrain drivetrain, Intake intake) {
        this.drivetrain = drivetrain;
        this.intake = intake;
    }
    public Command driveAndIntake() {
        return Commands.sequence(
            Commands.parallel(
                drivetrain.driveCommand(0.5, 0.5),
                intake.runIntakeCommand(1.0)
            ).withTimeout(5.0),
            Commands.parallel(
                drivetrain.stopCommand();
                intake.stopCommand();
            )
        );
    }
    public Command driveThenIntake() {
        return Commands.sequence(
            drivetrain.driveCommand(0.5, 0.5).withTimeout(5.0),
            drivetrain.stopCommand(),
            intake.runIntakeCommand(1.0).withTimeout(5.0),
            intake.stopCommand()
        );
    }
}
```

```

    });
  }
}

```

Then, elsewhere in our code, we can instantiate an single instance of this class and use it to produce several commands:

随后，可以在代码的其他位置实例化这个类一次，并用它生成多个 Command：

```

AutoRoutines autoRoutines = new AutoRoutines(this.drivetrain, this.intake);
Command driveAndIntake = autoRoutines.driveAndIntake();
Command driveThenIntake = autoRoutines.driveThenIntake();
Command drivingAndIntakingSequence = Commands.sequence(
    autoRoutines.driveAndIntake(),
    autoRoutines.driveThenIntake()
);

```

Capturing State in Inline Commands / 在内联 Command 中捕获状态

Inline commands are extremely concise and expressive, but do not offer explicit support for commands that have their own internal state (such as a drivetrain trajectory following command, which may encapsulate an entire controller). This is often accomplished by instead writing a Command class, which will be covered later in this article.

内联 Command 非常简洁且富于表达力，但对于具有自身内部状态的 Command（例如可能封装了完整控制器的底盘轨迹跟踪 Command），它并不提供显式支持。这通常通过改写一个 Command 类来实现，本文稍后会讨论。

However, it is still possible to ergonomically write a stateful command composition using inline syntax, so long as we are working within a factory method. To do so, we declare the state as a method local and “capture” it in our inline definition. For example, consider the following instance command factory to turn a drivetrain to a specific angle with a PID controller:

不过，只要在工厂方法中工作，仍可以用内联语法方便地编写有状态的 Command 组合。做法是将状态声明为方法局部变量，并在内联定义中“捕获”它。例如，考虑下面这个使用 PID 控制器将底盘转到指定角度的实例 Command 工厂：

NOTE: The `Subsystem.run` and `Subsystem.runOnce` factory methods sugar the creation of a `RunCommand` and an `InstantCommand` requiring this subsystem.

说明：`Subsystem.run` 和 `Subsystem.runOnce` 工厂方法分别简化了创建以 `this subsystem` 为需求资源的 `RunCommand` 和 `InstantCommand` 的过程。

```

public Command turnToAngle(double targetDegrees) {
    // Create a controller for the inline command to capture
    PIDController controller = new PIDController(Constants.kTurnToAngleP, 0, 0);
    // We can do whatever configuration we want on the created state before returning from the factory
    controller.setPositionTolerance(Constants.kTurnToAngleTolerance);
    // Try to turn at a rate proportional to the heading error until we're at the setpoint, then stop
    return run(() -> arcadeDrive(0, -controller.calculate(gyro.getHeading(), targetDegrees)))
        .until(controller::atSetpoint)
        .andThen(runOnce(() -> arcadeDrive(0, 0)));
}

```

This pattern works very well in Java so long as the captured state is “effectively final” - i.e., it is never reassigned. This means that we cannot directly define and capture primitive types (e.g. int, double, boolean) - to circumvent this, we need to wrap any state primitives in a mutable container type (the same way PIDController wraps its internal kP, kI, and kD values).

在 Java 中，只要捕获的状态是“实际上为 final”的（也就是从不重新赋值），这一模式就很有效。这意味着无法直接定义并捕获基本类型值（如 int、double、boolean）；为解决这个问题，需要把这些基本类型状态包装

到可变容器类型中（就像 `PIDController` 封装内部的 `kP`、`kI` 和 `kD` 值一样）。

译注：Java 可以捕获实际上为 `final` 的基本类型局部变量；需要可变容器的是必须在 `lambda` 中改变的状态。

Writing Command Classes / 编写 Command 类

Another possible way to define reusable commands is to write a class that represents the command. This is typically done by subclassing either `Command` or one of the `CommandGroup` classes.

定义可复用 `Command` 的另一种方式，是编写一个表示该 `Command` 的类。通常通过继承 `Command` 或某个 `CommandGroup` 类实现。

Subclassing Command / 继承 Command

Returning to our simple intake command from earlier, we could do this by creating a new subclass of `Command` that implements the necessary `initialize` and `end` methods.

回到前面那个简单的吸取机构 `Command`，可以创建一个新的 `Command` 子类，实现所需的 `initialize` 和 `end` 方法。

```
public class RunIntakeCommand extends Command {
    private Intake m_intake;
    public RunIntakeCommand(Intake intake) {
        this.m_intake = intake;
        addRequirements(intake);
    }
    @Override
    public void initialize() {
        m_intake.set(1.0);
    }
    @Override
    public void end(boolean interrupted) {
        m_intake.set(0.0);
    }
    // execute() defaults to do nothing
    // isFinished() defaults to return false
}
```

This, however, is just as cumbersome as the original repetitive code, if not more verbose. The only two lines that really matter in this entire file are the two calls to `intake.set()`, yet there are over 20 lines of boilerplate code! Not to mention, doing this for a lot of robot actions quickly clutters up a robot project with dozens of small files. Nevertheless, this might feel more “natural,” particularly for programmers who prefer to stick closely to an object-oriented model.

但这样至少和最初的重复代码一样繁琐，甚至更啰嗦。整个文件中真正重要的只有两行 `intake.set()` 调用，却有超过 20 行样板代码！更不用说，如果为许多机器人动作都这样做，项目很快就会被几十个小文件塞满。尽管如此，这对某些程序员可能感觉更“自然”，尤其是倾向于严格遵循面向对象模型的人。

This approach should be used for commands with internal state (not subsystem state!), as the class can have fields to manage said state. It may also be more intuitive to write commands with complex logic as classes, especially for those less experienced with command composition. As the command is detached from any specific subsystem class and the required subsystem objects are injected through the constructor, this approach deals well with commands involving multiple subsystems.

对于有内部状态（不是 subsystem 状态！）的 `Command`，应使用此方式，因为类可以通过字段管理该状态。对于复杂逻辑，编写 `Command` 类可能也更直观，尤其是对 `Command` 组合经验较少的程序员。由于 `Command` 独立于任何特定 subsystem 类，且所需 subsystem 对象通过构造函数注入，这种方式也适合涉及多个 subsystem 的 `Command`。

Subclassing Command Groups / 继承 Command 组

If we wish to write composite commands as their own classes, we may write a constructor-only subclass of the most exterior group type. For example, an intake-then-outtake sequence (with single-subsystem commands defined as instance factory methods) can look like this:

若希望将复合 Command 写成独立类，可以继承最外层的组类型，并让子类只包含构造函数。例如，一个先吸入、再排出的序列（其中单 subsystem Command 定义为实例工厂方法）可以写成这样：

```
public class IntakeThenOuttake extends SequentialCommandGroup {
    public IntakeThenOuttake(Intake intake) {
        super(
            intake.runIntakeCommand(1.0).withTimeout(2.0),
            new WaitCommand(2.0),
            intake.runIntakeCommand(-1).withTimeout(5.0)
        );
    }
}
```

This is relatively short and minimizes boilerplate. It is also comfortable to use in a purely object-oriented paradigm and may be more acceptable to novice programmers. However, it has some downsides. For one, it is not immediately clear exactly what type of command group this is from the constructor definition: it is better to define this in a more inline and expressive way, particularly when nested command groups start showing up. Additionally, it requires a new file for every single command group, even when the groups are conceptually related.

这种方式比较简短，也减少了样板代码。在纯面向对象范式中，它使用起来也很顺手，初学者可能更容易接受。不过，它也有缺点。首先，仅从构造函数定义不能立刻看出具体是哪种 Command 组；尤其当出现嵌套 Command 组时，用更内联、表达更清晰的方式定义会更好。此外，每个 Command 组都需要新建文件，即使这些组在概念上彼此相关。

As with factory methods, state can be defined and captured within the command group subclass constructor, if necessary.

与工厂方法一样，如有需要，也可以在 Command 组子类的构造函数中定义并捕获状态。

Summary / 总结

Approach 方式	Primary Use Case 主要适用场景	Single-subsystem Commands 单 subsystem Command	Multi-subsystem Commands 多 subsystem Command	Stateful Commands 有状态 Command	Complex Logic Commands 复杂逻辑 Command
Instance Factory Methods	Single-subsystem commands	Excels at them	No	Yes, but must obey capture rules	Yes
实例工厂方法	单 subsystem Command	非常适合	不适合	可以，但必须遵守捕 获规则	可以
Subclassing Command	Stateful commands	Very verbose	Relatively verbose	Excels at them	Yes; may be more natural than other approaches
继承 Command	有状态 Command	非常冗长	相对冗长	非常适合	可以；可能比其他方 式更自然
Static and Instance Command Factories	Multi-subsystem commands	Yes	Yes	Yes, but must obey capture rules	Yes
静态与实例 Command 工厂	多 subsystem Command	可以	可以	可以，但必须遵守捕 获规则	可以
Subclassing Command Groups	Multi-subsystem command groups	Yes	Yes	Yes, but must obey capture rules	Yes
继承 Command 组	多 subsystem Command 组	可以	可以	可以，但必须遵守捕 获规则	可以

08 The Command Scheduler / Command 调度器

The `CommandScheduler` ([Java](#)) is the class responsible for actually running commands. Each iteration (ordinarily once per 20ms), the scheduler polls all registered buttons, schedules commands for execution accordingly, runs the command bodies of all scheduled commands, and ends those commands that have finished or are interrupted.

`CommandScheduler` (Java) 是实际负责运行 `Command` 的类。每个周期（通常每 20ms 一次），调度器轮询所有已注册的按钮，据此调度 `Command` 执行，运行所有已调度 `Command` 的主体，并结束已完成或被中断的 `Command`。

The `CommandScheduler` also runs the `periodic()` method of each registered `Subsystem`.

`CommandScheduler` 还会运行每个已注册 `Subsystem` 的 `periodic()` 方法。

Using the Command Scheduler / 使用 Command 调度器

The `CommandScheduler` is a singleton, meaning that it is a globally-accessible class with only one instance. Accordingly, in order to access the scheduler, users must call the `CommandScheduler.getInstance()` command.

`CommandScheduler` 是一个单例，也就是全局可访问且只有一个实例的类。因此，要访问调度器，用户必须调用 `CommandScheduler.getInstance()`。

For the most part, users do not have to call scheduler methods directly - almost all important scheduler methods have convenience wrappers elsewhere (e.g. in the `Command` and `Subsystem` classes).

在多数情况下，用户不必直接调用调度器方法——几乎所有重要的调度器方法，都在别处有方便的封装（例如在 `Command` 和 `Subsystem` 类中）。

However, there is one exception: users must call `CommandScheduler.getInstance().run()` from the `robotPeriodic()` method of their `Robot` class. If this is not done, the scheduler will never run, and the command framework will not work. The provided command-based project template has this call already included.

但有一个例外：用户必须在其 `Robot` 类的 `robotPeriodic()` 方法中调用 `CommandScheduler.getInstance().run()`。如果不这样做，调度器就永远不会运行，`Command` 框架也无法工作。提供的 `Command-Based` 项目模板已包含这一调用。

The `schedule()` Method / `schedule()` 方法

To schedule a command, users call the `schedule()` method ([Java](#)). This method takes a command, and attempts to add it to list of currently-running commands, pending whether it is already running or whether its requirements are available. If it is added, its `initialize()` method is called.

要调度一个 `Command`，用户调用 `schedule()` 方法 (Java)。此方法接收一个 `Command`，并根据它是否已在运行及其需求资源是否可用，尝试将它加入当前运行的 `Command` 列表。若成功加入，就会调用它的 `initialize()` 方法。

This method walks through the following steps:

此方法依次执行以下步骤：

1. Verifies that the command isn't in a composition.

确认该 Command 不属于某个组合。

2. [No-op](#) if scheduler is disabled, command is already scheduled, or robot is disabled and command doesn't [runsWhenDisabled](#).

如果调度器已禁用、Command 已被调度，或者机器人处于禁用状态且该 Command 不允许在禁用时运行，则不执行任何操作。

3. If requirements are in use:

如果需求资源已被占用：

- If all conflicting commands are interruptible, cancel them.

若所有冲突的 Command 都允许被中断，则取消它们。

- If not, don't schedule the new command.

否则，不调度新的 Command。

4. Call `initialize()`.

调用 `initialize()`。

```
private void schedule(Command command) {
    if (command == null) {
        DriverStation.reportWarning("Tried to schedule a null command", true);
        return;
    }
    if (m_inRunLoop) {
        m_toSchedule.add(command);
        return;
    }
    requireNotComposed(command);
    // Do nothing if the scheduler is disabled, the robot is disabled and the command doesn't
    // run when disabled, or the command is already scheduled.
    if (m_disabled
        || isScheduled(command)
        || RobotState.isDisabled() && !command.runsWhenDisabled()) {
        return;
    }
    Set<Subsystem> requirements = command.getRequirements();
    // Schedule the command if the requirements are not currently in-use.
    if (Collections.disjoint(m_requirements.keySet(), requirements)) {
        initCommand(command, requirements);
    } else {
        // Else check if the requirements that are in use have all have interruptible commands,
        // and if so, interrupt those commands and schedule the new command.
        for (Subsystem requirement : requirements) {
            Command requiring = requiring(requirement);
            if (requiring != null
                && requiring.getInterruptionBehavior() == InterruptionBehavior.kCancelIncoming) {
                return;
            }
        }
        for (Subsystem requirement : requirements) {
            Command requiring = requiring(requirement);
            if (requiring != null) {
                cancel(requiring);
            }
        }
        initCommand(command, requirements);
    }
}
```

```
private void initCommand(Command command, Set<Subsystem> requirements) {
    m_scheduledCommands.add(command);
    for (Subsystem requirement : requirements) {
        m_requirements.put(requirement, command);
    }
    command.initialize();
    for (Consumer<Command> action : m_initActions) {
        action.accept(command);
    }
}
```

```
m_watchdog.addEpoch(command.getName() + ".initialize()");
```

The Scheduler Run Sequence / 调度器的运行顺序

NOTE: The `initialize()` method of each `Command` is called when the command is scheduled, which is not necessarily when the scheduler runs (unless that command is bound to a button).

说明：每个 `Command` 的 `initialize()` 方法会在该 `Command` 被调度时调用；这不一定发生在调度器运行时（除非该 `Command` 绑定到按钮）。

What does a single iteration of the scheduler's `run()` method ([Java](#)) actually do? The following section walks through the logic of a scheduler iteration. For the full implementation, see the source code ([Java](#)).

调度器的 `run()` 方法（Java）每运行一个周期，究竟会做什么？以下各节依次介绍一个调度周期中的逻辑。完整实现请参阅源码（Java）。

Step 1: Run Subsystem Periodic Methods / 步骤 1：运行 Subsystem 的周期方法

First, the scheduler runs the `periodic()` method of each registered `Subsystem`. In simulation, each subsystem's `simulationPeriodic()` method is called as well.

首先，调度器运行每个已注册 `Subsystem` 的 `periodic()` 方法。在仿真中，还会调用各 subsystem 的 `simulationPeriodic()` 方法。

```
// Run the periodic method of all registered subsystems.
for (Subsystem subsystem : m_subsystems.keySet()) {
    subsystem.periodic();
    if (RobotBase.isSimulation()) {
        subsystem.simulationPeriodic();
    }
    m_watchdog.addEpoch(subsystem.getClass().getSimpleName() + ".periodic()");
}
```

Step 2: Poll Command Scheduling Triggers / 步骤 2：轮询 Command 调度触发器

NOTE: For more information on how trigger bindings work, see [Binding Commands to Triggers](#)

说明：有关触发器绑定的工作原理，请参阅 [Binding Commands to Triggers](#)。

Secondly, the scheduler polls the state of all registered triggers to see if any new commands that have been bound to those triggers should be scheduled. If the conditions for scheduling a bound command are met, the command is scheduled and its `initialize()` method is run.

其次，调度器轮询所有已注册触发器的状态，检查是否应调度绑定在这些触发器上的新 `Command`。如果满足某个绑定 `Command` 的调度条件，就会调度该 `Command` 并运行其 `initialize()` 方法。

NOTE: If a newly-scheduled command has requirement conflicts with a currently-running command, the currently-running command is interrupted first. The `end(true)` method of the interrupted command is called before the `initialize()` method of the new command.

说明：如果新调度的 `Command` 与当前运行的 `Command` 存在需求资源冲突，会先中断当前运行的 `Command`。被中断 `Command` 的 `end(true)` 方法会在新 `Command` 的 `initialize()` 方法之前调用。

```
// Poll buttons for new commands to add.
loopCache.poll();
m_watchdog.addEpoch("buttons.run()");
```

Step 3: Run/Finish Scheduled Commands / 步骤 3: 运行 / 结束已调度的 Command

Thirdly, the scheduler calls the `execute()` method of each currently-scheduled command, and then checks whether the command has finished by calling the `isFinished()` method. If the command has finished, the `end()` method is also called, and the command is de-scheduled and its required subsystems are freed.

第三，调度器调用每个当前已调度 Command 的 `execute()` 方法，然后调用 `isFinished()` 方法检查该 Command 是否结束。如果已结束，还会调用 `end()` 方法，将 Command 从调度中移除，并释放其需求资源所对应的 subsystem。

Note that this sequence of calls is done in order for each command - thus, one command may have its `end()` method called before another has its `execute()` method called. Commands are handled in the order they were scheduled.

注意，这一调用顺序是分别对每个 Command 执行的——因此，某个 Command 的 `end()` 可能先于另一个 Command 的 `execute()` 被调用。Command 按调度顺序处理。

```
// Run scheduled commands, remove finished commands.
for (Iterator<Command> iterator = m_scheduledCommands.iterator(); iterator.hasNext(); ) {
    Command command = iterator.next();
    if (!command.runsWhenDisabled() && RobotState.isDisabled()) {
        command.end(true);
        for (Consumer<Command> action : m_interruptActions) {
            action.accept(command);
        }
        m_requirements.keySet().removeAll(command.getRequirements());
        iterator.remove();
        m_watchdog.addEpoch(command.getName() + ".end(true)");
        continue;
    }
    command.execute();
    for (Consumer<Command> action : m_executeActions) {
        action.accept(command);
    }
    m_watchdog.addEpoch(command.getName() + ".execute()");
    if (command.isFinished()) {
        command.end(false);
        for (Consumer<Command> action : m_finishActions) {
            action.accept(command);
        }
        iterator.remove();
        m_requirements.keySet().removeAll(command.getRequirements());
        m_watchdog.addEpoch(command.getName() + ".end(false)");
    }
}
```

Step 4: Schedule Default Commands / 步骤 4: 调度默认 Command

Finally, any registered `Subsystem` has its default command scheduled (if it has one). Note that the `initialize()` method of the default command will be called at this time.

最后，为任何已注册且有默认 Command 的 `Subsystem` 调度其默认 Command。注意，默认 Command 的 `initialize()` 方法此时会被调用。

```
// Add default commands for un-required registered subsystems.
for (Map.Entry<Subsystem, Command> subsystemCommand : m_subsystems.entrySet()) {
    if (!m_requirements.containsKey(subsystemCommand.getKey())
        && subsystemCommand.getValue() != null) {
        schedule(subsystemCommand.getValue());
    }
}
```

Disabling the Scheduler / 禁用调度器

The scheduler can be disabled by calling `CommandScheduler.getInstance().disable()`. When disabled, the scheduler's `schedule()` and `run()` commands will not do anything.

可以调用 `CommandScheduler.getInstance().disable()` 禁用调度器。禁用后，调度器的 `schedule()` 和 `run()` 命令不会执行任何操作。

The scheduler may be re-enabled by calling `CommandScheduler.getInstance().enable()`.

可以调用 `CommandScheduler.getInstance().enable()` 重新启用调度器。

Command Event Methods / Command 事件方法

Occasionally, it is desirable to have the scheduler execute a custom action whenever a certain command event (initialization, execution, or ending) occurs. This can be done with the following methods:

有时，需要调度器在某个 Command 事件（初始化、执行或结束）发生时执行自定义动作。可以使用以下方法：

- `onCommandInitialize` ([Java](#)) runs a specified action whenever a command is initialized.
每当一个 Command 初始化时，`onCommandInitialize` (Java) 运行指定动作。
- `onCommandExecute` ([Java](#)) runs a specified action whenever a command is executed.
每当一个 Command 执行时，`onCommandExecute` (Java) 运行指定动作。
- `onCommandFinish` ([Java](#)) runs a specified action whenever a command finishes normally (i.e. the `isFinished()` method returned true).
每当一个 Command 正常结束时（即 `isFinished()` 方法返回 true），`onCommandFinish` (Java) 运行指定动作。
- `onCommandInterrupt` ([Java](#)) runs a specified action whenever a command is interrupted (i.e. by being explicitly canceled or by another command that shares one of its requirements).
每当一个 Command 被中断时（例如被显式取消，或被另一个具有相同需求资源的 Command 中断），`onCommandInterrupt` (Java) 运行指定动作。

A typical use-case for these methods is adding markers in an event log whenever a command scheduling event takes place, as demonstrated in the following code from the HatchbotInlined example project ([Java](#)):

这些方法的一个典型用途，是在每次发生 Command 调度事件时向事件日志添加标记，如以下 HatchbotInlined 示例项目 (Java) 中的代码所示：

```
// Set the scheduler to log Shuffleboard events for command initialize, interrupt, finish
CommandScheduler.getInstance()
    .onCommandInitialize(
        command ->
            Shuffleboard.addEventMarker(
                "Command initialized", command.getName(), EventImportance.kNormal));
CommandScheduler.getInstance()
    .onCommandInterrupt(
        command ->
            Shuffleboard.addEventMarker(
                "Command interrupted", command.getName(), EventImportance.kNormal));
CommandScheduler.getInstance()
    .onCommandFinish(
        command ->
            Shuffleboard.addEventMarker(
                "Command finished", command.getName(), EventImportance.kNormal));
```

09 PID Control in Command-based / Command-Based 中的 PID 控制

NOTE: For a description of the WPILib PID control features used by these command-based wrappers, see [PID Control in WPILib](#).

说明：关于这些 Command-Based 封装所使用的 WPILib PID 控制功能，请参阅 PID Control in WPILib。

One of the most common control algorithms used in FRC® is the [PID](#) controller. WPILib offers its own [PIDController](#) class to help teams implement this functionality on their robots. The following example is from the RapidReactCommandBot example project ([Java](#)) and shows how PIDControllers can be used within the command-based framework:

PID 控制器是 FRC® 中最常用的控制算法之一。WPILib 提供了自己的 [PIDController](#) 类，帮助队伍在机器人上实现这一功能。以下示例来自 RapidReactCommandBot 示例项目（Java），展示了如何在 Command-Based 框架中使用 PIDController：

```
package edu.wpi.first.wpilibj.examples.rapidreactcommandbot.subsystems;
import static edu.wpi.first.wpilibj2.command.Commands.parallel;
import static edu.wpi.first.wpilibj2.command.Commands.waitUntil;
import edu.wpi.first.epilogue.Logged;
import edu.wpi.first.math.controller.PIDController;
import edu.wpi.first.math.controller.SimpleMotorFeedforward;
import edu.wpi.first.wpilibj.Encoder;
import edu.wpi.first.wpilibj.examples.rapidreactcommandbot.Constants.ShooterConstants;
import edu.wpi.first.wpilibj.motorcontrol.PWMSparkMax;
import edu.wpi.first.wpilibj2.command.Command;
import edu.wpi.first.wpilibj2.command.SubsystemBase;
@Logged
public class Shooter extends SubsystemBase {
    private final PWMSparkMax m_shooterMotor = new PWMSparkMax(ShooterConstants.kShooterMotorPort);
    private final PWMSparkMax m_feederMotor = new PWMSparkMax(ShooterConstants.kFeederMotorPort);
    private final Encoder m_shooterEncoder =
        new Encoder(
            ShooterConstants.kEncoderPorts[0],
            ShooterConstants.kEncoderPorts[1],
            ShooterConstants.kEncoderReversed);
    private final SimpleMotorFeedforward m_shooterFeedforward =
        new SimpleMotorFeedforward(
            ShooterConstants.kSVolts, ShooterConstants.kVVoltSecondsPerRotation);
    private final PIDController m_shooterFeedback = new PIDController(ShooterConstants.kP, 0.0, 0.0);
    /** The shooter subsystem for the robot. */
    public Shooter() {
        m_shooterFeedback.setTolerance(ShooterConstants.kShooterToleranceRPS);
        m_shooterEncoder.setDistancePerPulse(ShooterConstants.kEncoderDistancePerPulse);
        // Set default command to turn off both the shooter and feeder motors, and then idle
        setDefaultCommand(
            runOnce(
                () -> {
                    m_shooterMotor.disable();
                    m_feederMotor.disable();
                }
            ).andThen(run(() -> {})))
            .withName("Idle");
    }
    /**
     * Returns a command to shoot the balls currently stored in the robot. Spins the shooter flywheel
     * up to the specified setpoint, and then runs the feeder motor.
     *
     * @param setpointRotationsPerSecond The desired shooter velocity
     */
    public Command shootCommand(double setpointRotationsPerSecond) {
        return parallel(
            // Run the shooter flywheel at the desired setpoint using feedforward and feedback
            run(
                () -> {
                    m_shooterMotor.set(
                        m_shooterFeedforward.calculate(setpointRotationsPerSecond)
                        + m_shooterFeedback.calculate(
                            m_shooterEncoder.getRate(), setpointRotationsPerSecond));
                }
            )
        );
    }
}
```

```
    }},  
    // Wait until the shooter has reached the setpoint, and then run the feeder  
    waitUntil(m_shooterFeedback::atSetpoint).andThen(() -> m_feederMotor.set(1)))  
    .withName("Shoot");  
}  
}
```

A `PIDController` is declared inside the `Shooter` subsystem. It is used by `ShootCommand` alongside a feedforward to spin the shooter flywheel to the specified velocity. Once the `PIDController` reaches the specified velocity, the `ShootCommand` runs the feeder.

`Shooter` subsystem 内声明了一个 `PIDController`。`ShootCommand` 将它与前馈配合使用，使发射飞轮达到指定速度。当 `PIDController` 达到指定速度后，`ShootCommand` 启动送料机构。

10 Motion Profiling in Command-based / Command-Based 中的运动曲线规划

NOTE: For a description of the WPILib motion profiling features used by these command-based wrappers, see [Trapezoidal Motion Profiles in WPILib](#).

说明：关于这些 Command-Based 封装所使用的 WPILib 运动曲线规划功能，请参阅 Trapezoidal Motion Profiles in WPILib。

NOTE: The `TrapezoidProfile` class, used on its own, is most useful when composed with external controllers, such as a “smart” motor controller with a built-in PID functionality. For combining trapezoidal motion profiling with WPILib’s `PIDController`, see [Combining Motion Profiling and PID in Command-Based](#).

说明：单独使用 `TrapezoidProfile` 类时，最适合与外部控制器组合，例如内置 PID 功能的“智能”电机控制器。有关将梯形运动曲线规划与 WPILib 的 `PIDController` 结合使用的方法，请参阅 [Combining Motion Profiling and PID in Command-Based](#)。

When controlling a mechanism, it is often desirable to move it smoothly between two positions, rather than to abruptly change its setpoint. This is called “motion-profiling,” and is supported in WPILib through the `TrapezoidProfile` class ([Java](#)).

控制机构时，通常希望它在两个位置之间平稳运动，而不是突然改变其 setpoint。这称为“运动曲线规划”（motion profiling），WPILib 通过 `TrapezoidProfile` 类提供支持（Java）。

The following examples are taken from the `DriveDistanceOffboard` example project ([Java](#)):

以下示例取自 `DriveDistanceOffboard` 示例项目（Java）：

```
package edu.wpi.first.wpilibj.examples.drivedistanceoffboard.subsystems;
import edu.wpi.first.math.controller.SimpleMotorFeedforward;
import edu.wpi.first.math.trajectory.TrapezoidProfile;
import edu.wpi.first.math.trajectory.TrapezoidProfile.State;
import edu.wpi.first.util.sendable.SendableRegistry;
import edu.wpi.first.wpilibj.RobotController;
import edu.wpi.first.wpilibj.Timer;
import edu.wpi.first.wpilibj.drive.DifferentialDrive;
import edu.wpi.first.wpilibj.examples.drivedistanceoffboard.Constants.DriveConstants;
import edu.wpi.first.wpilibj.examples.drivedistanceoffboard.ExampleSmartMotorController;
import edu.wpi.first.wpilibj2.command.Command;
import edu.wpi.first.wpilibj2.command.SubsystemBase;
public class DriveSubsystem extends SubsystemBase {
    // The motors on the left side of the drive.
    private final ExampleSmartMotorController m_leftLeader =
        new ExampleSmartMotorController(DriveConstants.kLeftMotor1Port);
    private final ExampleSmartMotorController m_leftFollower =
        new ExampleSmartMotorController(DriveConstants.kLeftMotor2Port);
    // The motors on the right side of the drive.
    private final ExampleSmartMotorController m_rightLeader =
        new ExampleSmartMotorController(DriveConstants.kRightMotor1Port);
    private final ExampleSmartMotorController m_rightFollower =
        new ExampleSmartMotorController(DriveConstants.kRightMotor2Port);
    // The feedforward controller.
    private final SimpleMotorFeedforward m_feedforward =
        new SimpleMotorFeedforward(
            DriveConstants.ksVolts,
            DriveConstants.kvVoltSecondsPerMeter,
            DriveConstants.kaVoltSecondsSquaredPerMeter);
    // The robot's drive
    private final DifferentialDrive m_drive =
        new DifferentialDrive(m_leftLeader::set, m_rightLeader::set);
    // The trapezoid profile
    private final TrapezoidProfile m_profile =
        new TrapezoidProfile(
            new TrapezoidProfile.Constraints(
```



```

        DriveConstants.kMaxSpeedMetersPerSecond,
        DriveConstants.kMaxAccelerationMetersPerSecondSquared));
// The timer
private final Timer m_timer = new Timer();
/** Creates a new DriveSubsystem. */
public DriveSubsystem() {
    SendableRegistry.addChild(m_drive, m_leftLeader);
    SendableRegistry.addChild(m_drive, m_rightLeader);
    // We need to invert one side of the drivetrain so that positive voltages
    // result in both sides moving forward. Depending on how your robot's
    // gearbox is constructed, you might have to invert the left side instead.
    m_rightLeader.setInverted(true);
    m_leftFollower.follow(m_leftLeader);
    m_rightFollower.follow(m_rightLeader);
    m_leftLeader.setPID(DriveConstants.kp, 0, 0);
    m_rightLeader.setPID(DriveConstants.kp, 0, 0);
}
/**
 * Drives the robot using arcade controls.
 *
 * @param fwd the commanded forward movement
 * @param rot the commanded rotation
 */
public void arcadeDrive(double fwd, double rot) {
    m_drive.arcadeDrive(fwd, rot);
}
/**
 * Attempts to follow the given drive states using offboard PID.
 *
 * @param currentLeft The current left wheel state.
 * @param currentRight The current right wheel state.
 * @param nextLeft The next left wheel state.
 * @param nextRight The next right wheel state.
 */
public void setDriveStates(
    TrapezoidProfile.State currentLeft,
    TrapezoidProfile.State currentRight,
    TrapezoidProfile.State nextLeft,
    TrapezoidProfile.State nextRight) {
    // Feedforward is divided by battery voltage to normalize it to [-1, 1]
    m_leftLeader.setSetpoint(
        ExampleSmartMotorController.PIDMode.kPosition,
        currentLeft.position,
        m_feedforward.calculateWithVelocities(currentLeft.velocity, nextLeft.velocity)
        / RobotController.getBatteryVoltage());
    m_rightLeader.setSetpoint(
        ExampleSmartMotorController.PIDMode.kPosition,
        currentRight.position,
        m_feedforward.calculateWithVelocities(currentLeft.velocity, nextLeft.velocity)
        / RobotController.getBatteryVoltage());
}
/**
 * Returns the left encoder distance.
 *
 * @return the left encoder distance
 */
public double getLeftEncoderDistance() {
    return m_leftLeader.getEncoderDistance();
}
/**
 * Returns the right encoder distance.
 *
 * @return the right encoder distance
 */
public double getRightEncoderDistance() {
    return m_rightLeader.getEncoderDistance();
}
/** Resets the drive encoders. */
public void resetEncoders() {
    m_leftLeader.resetEncoder();
    m_rightLeader.resetEncoder();
}
/**
 * Sets the max output of the drive. Useful for scaling the drive to drive more slowly.
 *
 * @param maxOutput the maximum output to which the drive will be constrained
 */
public void setMaxOutput(double maxOutput) {
    m_drive.setMaxOutput(maxOutput);
}
/**

```

```

    * Creates a command to drive forward a specified distance using a motion profile.
    *
    * @param distance The distance to drive forward.
    * @return A command.
    */
    public Command profiledDriveDistance(double distance) {
        return startRun(
            () -> {
                // Restart timer so profile setpoints start at the beginning
                m_timer.restart();
                resetEncoders();
            },
            () -> {
                // Current state never changes, so we need to use a timer to get the setpoints we need
                // to be at
                var currentTime = m_timer.get();
                var currentSetpoint =
                    m_profile.calculate(currentTime, new State(), new State(distance, 0));
                var nextSetpoint =
                    m_profile.calculate(
                        currentTime + DriveConstants.kDt, new State(), new State(distance, 0));
                setDriveStates(currentSetpoint, currentSetpoint, nextSetpoint, nextSetpoint);
            })
            .until(() -> m_profile.isFinished(0));
    }
    private double m_initialLeftDistance;
    private double m_initialRightDistance;
    /**
     * Creates a command to drive forward a specified distance using a motion profile without
     * resetting the encoders.
     *
     * @param distance The distance to drive forward.
     * @return A command.
     */
    public Command dynamicProfiledDriveDistance(double distance) {
        return startRun(
            () -> {
                // Restart timer so profile setpoints start at the beginning
                m_timer.restart();
                // Store distance so we know the target distance for each encoder
                m_initialLeftDistance = getLeftEncoderDistance();
                m_initialRightDistance = getRightEncoderDistance();
            },
            () -> {
                // Current state never changes for the duration of the command, so we need to use a
                // timer to get the setpoints we need to be at
                var currentTime = m_timer.get();
                var currentLeftSetpoint =
                    m_profile.calculate(
                        currentTime,
                        new State(m_initialLeftDistance, 0),
                        new State(m_initialLeftDistance + distance, 0));
                var currentRightSetpoint =
                    m_profile.calculate(
                        currentTime,
                        new State(m_initialRightDistance, 0),
                        new State(m_initialRightDistance + distance, 0));
                var nextLeftSetpoint =
                    m_profile.calculate(
                        currentTime + DriveConstants.kDt,
                        new State(m_initialLeftDistance, 0),
                        new State(m_initialLeftDistance + distance, 0));
                var nextRightSetpoint =
                    m_profile.calculate(
                        currentTime + DriveConstants.kDt,
                        new State(m_initialRightDistance, 0),
                        new State(m_initialRightDistance + distance, 0));
                setDriveStates(
                    currentLeftSetpoint, currentRightSetpoint, nextLeftSetpoint, nextRightSetpoint);
            })
            .until(() -> m_profile.isFinished(0));
    }
}

```

There are two commands in this example. They function very similarly, with the main difference being that one resets encoders, and the other doesn't, which allows encoder data to be preserved.

本示例中有两个 Command。它们的功能非常相似，主要区别在于其中一个会重置编码器，另一个不会，因此可以保留编码器数据。

The subsystem contains a `TrapezoidProfile` with a `Timer`. The timer is used along with a `kDt` constant of 0.02 seconds to calculate the current and next states from the `TrapezoidProfile`. The current state is fed to the “smart” motor controller for PID control, while the current and next state are used to calculate feedforward outputs. Both commands end when `isFinished(0)` returns true, which means that the profile has reached the goal state.

该 subsystem 包含一个 `TrapezoidProfile` 和一个 `Timer`。计时器与 0.02 秒的 `kDt` 常量一起用于计算 `TrapezoidProfile` 的当前状态和下一状态。当前状态会送入 “智能” 电机控制器进行 PID 控制，而当前状态和下一状态都会用于计算前馈输出。当 `isFinished(0)` 返回 true 时，两个 Command 都会结束，这表示运动曲线已到达目标状态。

11 Combining Motion Profiling and PID in Command-Based

/ 在 Command-Based 中结合运动曲线规划与 PID

NOTE: For a description of the WPILib PID control features used by these command-based wrappers, see [PID Control in WPILib](#).

说明：关于这些 Command-Based 封装所使用的 WPILib PID 控制功能，请参阅 [PID Control in WPILib](#)。

A common FRC® controls solution is to pair a trapezoidal motion profile for setpoint generation with a PID controller for setpoint tracking. To facilitate this, WPILib includes its own [ProfiledPIDController](#) class. The following example is from the [RapidReactCommandBot](#) example project ([Java](#)) and shows how [ProfiledPIDController](#) can be used within the command-based framework to turn a drivetrain to a specified angle:

FRC® 中一种常见的控制方案，是使用梯形运动曲线生成 setpoint，并用 PID 控制器跟踪 setpoint。为此，WPILib 提供了自己的 [ProfiledPIDController](#) 类。以下示例来自 [RapidReactCommandBot](#) 示例项目（Java），展示了如何在 Command-Based 框架中使用 [ProfiledPIDController](#) 将底盘转向指定角度：

```
package edu.wpi.first.wpilibj.examples.rapidreactcommandbot.subsystems;
import edu.wpi.first.epilogue.Logged;
import edu.wpi.first.epilogue.NotLogged;
import edu.wpi.first.math.controller.ProfiledPIDController;
import edu.wpi.first.math.controller.SimpleMotorFeedforward;
import edu.wpi.first.math.trajetory.TrapezoidProfile;
import edu.wpi.first.util.sendable.SendableRegistry;
import edu.wpi.first.wpilibj.ADXRS450_Gyro;
import edu.wpi.first.wpilibj.Encoder;
import edu.wpi.first.wpilibj.RobotController;
import edu.wpi.first.wpilibj.drive.DifferentialDrive;
import edu.wpi.first.wpilibj.examples.rapidreactcommandbot.Constants.DriveConstants;
import edu.wpi.first.wpilibj.motorcontrol.PWMSparkMax;
import edu.wpi.first.wpilibj2.command.Command;
import edu.wpi.first.wpilibj2.command.SubsystemBase;
import java.util.function.DoubleSupplier;
@Logged
public class Drive extends SubsystemBase {
    // The motors on the left side of the drive.
    private final PWMSparkMax m_leftLeader = new PWMSparkMax(DriveConstants.kLeftMotor1Port);
    private final PWMSparkMax m_leftFollower = new PWMSparkMax(DriveConstants.kLeftMotor2Port);
    // The motors on the right side of the drive.
    private final PWMSparkMax m_rightLeader = new PWMSparkMax(DriveConstants.kRightMotor1Port);
    private final PWMSparkMax m_rightFollower = new PWMSparkMax(DriveConstants.kRightMotor2Port);
    // The robot's drive
    @NotLogged // Would duplicate motor data, there's no point sending it twice
    private final DifferentialDrive m_drive =
        new DifferentialDrive(m_leftLeader::set, m_rightLeader::set);
    // The left-side drive encoder
    private final Encoder m_leftEncoder =
        new Encoder(
            DriveConstants.kLeftEncoderPorts[0],
            DriveConstants.kLeftEncoderPorts[1],
            DriveConstants.kLeftEncoderReversed);
    // The right-side drive encoder
    private final Encoder m_rightEncoder =
        new Encoder(
            DriveConstants.kRightEncoderPorts[0],
            DriveConstants.kRightEncoderPorts[1],
            DriveConstants.kRightEncoderReversed);
    private final ADXRS450_Gyro m_gyro = new ADXRS450_Gyro();
    private final ProfiledPIDController m_controller =
        new ProfiledPIDController(
            DriveConstants.kTurnP,
            DriveConstants.kTurnI,
            DriveConstants.kTurnD,
            new TrapezoidProfile.Constraints(
                DriveConstants.kMaxTurnRateDegPerS,
                DriveConstants.kMaxTurnAccelerationDegPerSSquared));
    private final SimpleMotorFeedforward m_feedforward =
        new SimpleMotorFeedforward(
```

```

        DriveConstants.kVolts,
        DriveConstants.kVoltSecondsPerDegree,
        DriveConstants.kAVoltSecondsSquaredPerDegree);
/** Creates a new Drive subsystem. */
public Drive() {
    SendableRegistry.addChild(m_drive, m_leftLeader);
    SendableRegistry.addChild(m_drive, m_rightLeader);
    m_leftLeader.addFollower(m_leftFollower);
    m_rightLeader.addFollower(m_rightFollower);
    // We need to invert one side of the drivetrain so that positive voltages
    // result in both sides moving forward. Depending on how your robot's
    // gearbox is constructed, you might have to invert the left side instead.
    m_rightLeader.setInverted(true);
    // Sets the distance per pulse for the encoders
    m_leftEncoder.setDistancePerPulse(DriveConstants.kEncoderDistancePerPulse);
    m_rightEncoder.setDistancePerPulse(DriveConstants.kEncoderDistancePerPulse);
    // Set the controller to be continuous (because it is an angle controller)
    m_controller.enableContinuousInput(-180, 180);
    // Set the controller tolerance - the delta tolerance ensures the robot is stationary at the
    // setpoint before it is considered as having reached the reference
    m_controller.setTolerance(
        DriveConstants.kTurnToleranceDeg, DriveConstants.kTurnRateToleranceDegPerS);
}
/**
 * Returns a command that drives the robot with arcade controls.
 *
 * @param fwd the commanded forward movement
 * @param rot the commanded rotation
 */
public Command arcadeDriveCommand(DoubleSupplier fwd, DoubleSupplier rot) {
    // A split-stick arcade command, with forward/backward controlled by the left
    // hand, and turning controlled by the right.
    return run(() -> m_drive.arcadeDrive(fwd.getAsDouble(), rot.getAsDouble()))
        .withName("arcadeDrive");
}
/**
 * Returns a command that drives the robot forward a specified distance at a specified speed.
 *
 * @param distanceMeters The distance to drive forward in meters
 * @param speed The fraction of max speed at which to drive
 */
public Command driveDistanceCommand(double distanceMeters, double speed) {
    return runOnce(
        () -> {
            // Reset encoders at the start of the command
            m_leftEncoder.reset();
            m_rightEncoder.reset();
        })
        // Drive forward at specified speed
        .andThen(run(() -> m_drive.arcadeDrive(speed, 0)))
        // End command when we've traveled the specified distance
        .until(
            () ->
                Math.max(m_leftEncoder.getDistance(), m_rightEncoder.getDistance())
                    >= distanceMeters)
        // Stop the drive when the command ends
        .finallyDo(interrupted -> m_drive.stopMotor());
}
/**
 * Returns a command that turns the robot to the specified angle using a motion profile and PID
 * controller.
 *
 * @param angleDeg The angle to turn to
 */
public Command turnToAngleCommand(double angleDeg) {
    return startRun(
        () -> m_controller.reset(m_gyro.getRotation2d().getDegrees()),
        () ->
            m_drive.arcadeDrive(
                0,
                m_controller.calculate(m_gyro.getRotation2d().getDegrees(), angleDeg)
                    // Divide feedforward voltage by battery voltage to normalize it to [-1, 1]
                    + m_feedforward.calculate(m_controller.getSetpoint().velocity)
                    / RobotController.getBatteryVoltage()))
        .until(m_controller::atGoal)
        .finallyDo(() -> m_drive.arcadeDrive(0, 0));
}
}

```

`turnToAngleCommand` uses a `ProfiledPIDController` to smoothly turn the drivetrain. The `startRun` command factory is used to reset the `ProfiledPIDController` when the command is scheduled to avoid unwanted behavior, and to calculate PID and feedforward outputs to pass into the `arcadeDrive` method in order to drive the robot. The command is decorated using the `until` decorator to end the command when the `ProfiledPIDController` is finished with the profile. To ensure the drivetrain stops when the command ends, the `finallyDo` decorator is used to stop the drivetrain by setting the speed to zero.

`turnToAngleCommand` 使用 `ProfiledPIDController` 平稳地转动底盘。`startRun` Command 工厂用于在 Command 被调度时重置 `ProfiledPIDController`，以避免不期望的行为，并计算传给 `arcadeDrive` 方法的 PID 与前馈输出，从而驱动机器人。该 Command 使用 `until` 装饰器，在 `ProfiledPIDController` 完成运动曲线时结束。为确保 Command 结束时底盘停止，还使用 `finallyDo` 装饰器将速度设置为零。